

Also available as



Book

The 8051

MICROCONTROLLER Based
EMBEDDED SYSTEMS

Manish K Patel

**Mc
Graw
Hill
Education**

The 8051 Microcontroller based Embedded Systems

ABOUT THE AUTHOR



Manish K Patel completed his BE in Electronics and Communication Engineering in 2002 from G H Patel College of Engineering and Technology, Vallabh Vidyanagar, and ME in Electronics and Communication Systems in 2009 from Dharmsinh Desai University, Nadiad. At present, he is working as Assistant Professor in the Department of Electronics and Communication, Faculty of Technology, Dharmsinh Desai University, Nadiad, Gujarat. In his 12 years of teaching experience, he has carried out and guided many projects based on microcontrollers. His area of interest is design and implementation of distributed embedded systems.

The 8051 Microcontroller based Embedded Systems

Manish K Patel

*Dharmsinh Desai University Nadiad
Gujarat*



McGraw Hill Education (India) Private Limited
NEW DELHI

McGraw Hill Education Offices

New Delhi New York St Louis San Francisco Auckland Bogotá Caracas
Kuala Lumpur Lisbon London Madrid Mexico City Milan Montreal
San Juan Santiago Singapore Sydney Tokyo Toronto



McGraw Hill Education (India) Private Limited

Published by McGraw Hill Education (India) Private Limited
P-24, Green Park Extension, New Delhi 110016

The 8051 Microcontroller based Embedded Systems

Copyright © 2014 by McGraw Hill Education (India) Private Limited.

No part of this publication can be reproduced or distributed in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise or stored in a database or retrieval system without the prior written permission of the publishers. The program listings (if any) may be entered, stored and executed in a computer system, but they may not be reproduced for publication.

This edition can be exported from India only by the publishers,
McGraw Hill Education (India) Private Limited

ISBN (13) : 978-93-329-0125-4

ISBN (10) : 93-329-0125-2

Managing Director: *Kaushik Bellani*

Head—Higher Education (Publishing and Marketing): *Vibha Mahajan*

Senior Publishing Manager (SEM & Tech. Ed.): *Shalini Jha*

Associate Sponsoring Editor: *Smruti Snigdha*

Senior Editorial Researcher: *Amiya Mahapatra*

Manager—Production Systems: *Satinder S Baveja*

Assistant Manager—Editorial Services: *Sohini Mukherjee*

Senior Manager—Production: *P L Pandita*

Assistant General Manager (Marketing)—Higher Education: *Vijay Sarathi*

Senior Product Specialist: *Tina Jajoriya*

Senior Graphic Designer—Cover: *Meenu Raghav*

General Manager—Production: *Rajender P Ghansela*

Manager—Production: *Reji Kumar*

Information contained in this work has been obtained by McGraw Hill Education (India), from sources believed to be reliable. However, neither McGraw Hill Education (India) nor its authors guarantee the accuracy or completeness of any information published herein, and neither McGraw Hill Education (India) nor its authors shall be responsible for any errors, omissions, or damages arising out of use of this information. This work is published with the understanding that McGraw Hill Education (India) and its authors are supplying information but are not attempting to render engineering or other professional services. If such services are required, the assistance of an appropriate professional should be sought.

Typeset at Print-O-World, 2579, Mandir Lane, Shadipur, New Delhi 110 008, and printed at

Cover Printer :

Contents

Preface

xiii

1. Introduction to Microcontrollers 1

- 1.1 Computer System 2
 - 1.1.1 Central Processing Unit 2
 - CPU Components 2
 - 1.1.2 Memory 3
 - 1.1.3 I/O Unit 3
 - 1.1.4 System Bus 4
 - 1.1.5 How does the CPU Read Data from the Memory Chip? 5
 - 1.1.6 What can a Computer do? 5
 - 1. Data Movement 5
 - 2. Performing Binary Operations 5
 - 1.1.7 How does Computer Execute Program Instructions? 5
- 1.2 Microprocessor, Microcomputer and Microcontroller 5
 - 1.2.1 Microprocessor 5
 - 1.2.2 Microcomputer 6
 - 1.2.3 Microcontroller 6
 - 1.2.4 Comparison between Microprocessor and Microcontroller 7
- 1.3 Classification of Microcontrollers 7
 - 1.3.1 Word Length: 4, 8, 16, 32, 64-bit Microcontrollers 7
 - 1.3.2 Memory Architecture: Von Neumann and Harvard Architectures 8
 - 1. Von Neumann Architecture 8
 - 2. Harvard Architecture 8
 - 1.3.3 Core Architecture: Microcoded and Hardwired Designs 8
 - 1. Hardwired Design 8
 - 2. Microcoded Design 8
 - 1.3.4 Instruction Set Architectures: CISC and RISC 9
 - 1. CISC: Complex Instruction Set Computer 9
 - 2. RISC: Reduced Instruction Set Computer 9
- 1.4 Choosing a Microcontroller 10
- 1.5 Applications of Microcontrollers 10
- 1.6 History and Introduction to The 8051 Microcontroller Family 11
- 1.7 Overview of The 8051 Family 11
 - 1.7.1 Features of the 8051 (MCS 51) Family 11
 - 1.7.2 8051 Variants and Enhancements 12

1.7.3 Comparison between MCS 51, PIC, AVR and HCS11/12 Families 13

1.7.4 Advantages of using 8051 Family of Microcontrollers 13

1.8 Embedded Systems 15

- Embedded Microcontrollers 15

Points to Remember 15

Objective Questions 16

Review Questions with Answers 17

Exercise 18

2. Programming Model and Architecture of The 8051 19

- 2.1 The 8051 Architecture 20
 - 1. ALU 21
 - 2. Memory 21
 - 3. Peripherals 21
 - 4. Timing and Control Unit 21
 - 5. Oscillator 21
- 2.2 Programming Model of The 8051 21
- 2.3 On-Chip Memory Organization 22
 - 1. Special Function Registers (SFRs) 22
 - 2. Internal RAM 23
 - 3. Internal ROM 23
- 2.3.1 Special Function Registers (SFRs) 23
 - 1. Accumulator: A 23
 - 2. B 23
 - 3. PSW 23
 - 4. Program Counter: PC 24
 - 5. Data Pointer: DPTR 24
 - 6. Stack Pointer: SP 25
 - 7. I/O Port Registers (latches): P0, P1, P2 and P3 25
 - 8. Peripheral Data Registers: TL0, TH0, TL1, TH1, and SBUF 25
 - 9. Peripheral Control Registers: IP, IE, TMOD, TCON, SCON, and PCON 25
- 2.3.2 Internal RAM 25
 - 1. Register Banks 25
 - 2. Bit Addressable Memory 27
 - 3. General-Purpose RAM 28
- 2.3.3 Internal ROM 29
- 2.4 External Memory Organization 29
- Points to Remember 30*
- Objective Questions 31*
- Review Questions with Answers 32*
- Exercise 33*

3. Program Development Process and Tools 34

- 3.1 Programming Language 35
 - 3.1.1 Machine Language 35
 - 3.1.2 Assembly Language 35
 - 3.1.3 High-Level Language 35
 - 3.1.4 Comparison between Programming Languages 36
 - 3.1.5 Why Assembly Language? 36
 - 3.2 Assembly Language Structure 37
 - 3.2.1 Label 37
 - 3.2.2 Instructions 37
 - Instruction Size 37
 - 3.2.3 Comments 38
 - 3.3 Assembly-Language-Program Example 38
 - 3.4 Program Execution Process 39
 - 3.5 Software and Hardware Development Tools 39
 - 3.5.1 Design and Documentation Tools 39
 - 1. Flowcharts 40
 - 2. Pseudo-codes 40
 - 3.5.2 Software Development Tools 40
 - 1. Editor 40
 - 2. Assembler 41
 - 3. Cross Assembler 42
 - 4. Compiler 42
 - 5. Linker 42
 - 6. Simulator 42
 - 7. Debugger 42
 - 3.5.3 Hardware Development Tools 43
 - 1. Emulator or In Circuit Emulator (ICE) 43
 - 2. Logic Analyzer 43
 - 3. Target System 43
 - 3.6 Integrated Development Environments (IDE) 43
 - 3.7 Assembler Directives 43
 - 1. Originate-ORG 43
 - 2. Define Byte-DB 44
 - 3. Define Word-DW 44
 - 4. Equate-EQU 44
 - 5. END 44
 - 3.8 Program Development Process 45
 - 1. Create Source Code 45
 - 2. Assemble all Source Code Files 45
 - 3. Link all the Object Code Files 45
 - 4. Test the Program for Correctness 46
 - 3.9 Loading Program into Microcontroller 47
 - 1. Parallel Programming 47
 - 2. Serial Programming 47
 - 3.10 The Intel Hex File Format 48
 - Introduction to the Instruction Set 54
- Points to Remember 49*
Objective Questions 50
Review Questions with Answers 51
Exercise 52

4. Addressing Modes and Data Movement Instructions 53

- 4.1 Why Data Movement Instructions First? 54
 - 4.2 Addressing Modes 54
 - Acronyms used in the Instructions 55
 - 4.2.1 Immediate Addressing Mode 55
 - 1. Notations for Numbers of Different Number Systems 56
 - 2. Use of Expressions 56
 - 4.2.2 Register Addressing Mode 56
 - 4.2.3 Direct Addressing Mode 57
 - 4.2.4 Indirect Addressing Mode 58
 - 1. Register Indirect Addressing Mode 58
 - 2. Indexed Addressing Mode 59
 - 4.3 Operand Modifiers: # and @ 60
 - 4.4 External Memory Data Movements 61
 - 4.4.1 Data Memory Access 61
 - 4.4.2 Program Memory Access 62
 - 4.5 Data Exchange 63
 - 4.6 Push and Pop Instructions 63
- Points to Remember 65*
Objective Questions 67
Review Questions with Answers 68
Exercise 69

5. Arithmetic and Logical Instructions 70

- 5.1 Arithmetic Instructions 71
 - 5.1.1 Addition 71
 - Addition of Multi-Byte Numbers 72
 - 5.1.2 Subtraction 74
 - 5.1.3 Signed Arithmetic 75
 - 1. Positive Numbers 75
 - 2. Negative Numbers 75
 - 3. Overflow 77
 - 4. Addition of Unlike Signed Numbers 77
 - 5. Addition of Like Signed Numbers 78
 - 6. Subtraction of Unlike Signed Numbers 80
 - 7. Subtraction of Like Signed Numbers 81
 - 8. Recovering a Result from Overflow 82
 - 5.1.4 Decimal (Binary Coded Decimal—BCD) Arithmetic 83
 - DA A Decimal Adjust Accumulator for Addition 84
 - 5.1.5 Multiplication 85
 - Use of OV in Multiplication 85
 - 5.1.6 Division 86
 - Use of OV in Division 86
 - 5.1.7 Increment and Decrement 86
- 5.2 Logical Instructions 89
 - 5.2.1 Byte Operations 89
 - 1. AND Operation 89
 - 2. OR Operation 90

3. EX-OR Operation	90
4. Logical Operations with Ports	91
5.2.2 Unary Operations	92
3. Summary of Arithmetic and Logical Instructions	94
<i>Points to Remember</i>	95
<i>Objective Questions</i>	96
<i>Review Questions with Answers</i>	97
<i>Exercise</i>	98
6. Bit-Processing Instructions	99
6.1 Bit Addressability	100
6.2 Bit-Addressable Memories	100
6.2.1 Bit-Addressable Internal RAM	100
6.2.2 Bit-Addressable Special Function Registers	101
6.3 Bit-Processing Instructions	104
6.3.1 Instruction using Carry	104
6.3.2 Other Instructions	105
6.3.3 Conditional Jump Instructions	107
Summary of Bit-Processing Instructions	108
<i>Points to Remember</i>	108
<i>Objective Questions</i>	109
<i>Review Questions with Answers</i>	109
<i>Exercise</i>	110
7. Program-Flow Control Instructions	111
7.1 Jump Instructions	112
7.1.1 Unconditional Jumps	112
1. Short Jump	112
2. Absolute Jump	114
3. Long Jump	115
7.1.2 Conditional Jumps	117
1. Bit Jumps	117
2. Byte Jumps	118
7.2 Loops	119
Nested Loops	121
7.3 Calls and Subroutines	122
1. Relation of Calls and Stack	122
2. Cautions while Developing Subroutines	125
7.4 Stack Initialization and Overflow	127
7.5 Time-Delay Generation using Software NOP (No Operation)	127
Summary of Program-Flow Control Instructions	129
<i>Points to Remember</i>	130
<i>Objective Questions</i>	130
<i>Review Questions with Answers</i>	131
<i>Exercise</i>	133
8. Look-Up Tables and Jump Tables	134
8.1 Look-Up Tables and their Usage	135
8.2 Faster Evaluation of Functions	135
8.3 Miscellaneous Conversions	136
8.4 The 8051 and Look-Up Tables	136
Without using Look-up Table	140
With Using Look-up Table	140
8.5 Jump Tables	140
<i>Points to Remember</i>	141
<i>Objective Questions</i>	141
<i>Exercise</i>	142
9. Code Conversions, Array Processing and 16 Bit Arithmetic	143
9.1 Code Conversions	144
9.2 Array Processing	148
9.3 16 Bit Operations	153
9.4 Other Programs	155
<i>Points to Remember</i>	158
<i>Exercise</i>	158
10. Timing and Instruction Execution	159
10.1 The Clock Pulse	160
10.2 Machine Cycle	160
10.3 Instructions Timing	160
10.3.1 1 Byte–1 Machine Cycle Instructions	161
10.3.2 2 Byte–1 Machine Cycle Instructions	161
10.3.3 1 Byte–2 Machine Cycle Instructions	161
10.3.4 2 Byte–2 Machine Cycle Instructions	162
10.4 External Memory Access	162
10.4.1 External RAM (Data Memory) Access	162
10.4.2 Data Memory Read/Write Cycle	163
10.4.3 External ROM (Code Memory) Access	164
10.5 8051 Instruction Execution	165
10.5.1 MOV A, Operand	165
10.5.2 ADD A, Operand	166
10.5.3 LCALL Label	169
10.5.4 MOVX A, @ DPTR	169
<i>Points to Remember</i>	169
<i>Objective Questions</i>	171
<i>Review Questions with Answers</i>	171
<i>Exercise</i>	172
11. The 8051 Hardware, System Design and Troubleshooting	173
11.1 The 8051 Pin Diagram	174
11.2 The 8051 Pin Description	174
11.3 Power Consumption Control of The 8051	178
1. Idle Mode	178
2. Power Down Mode	178
Tip for Power Saving	178
11.4 Design of The 8051 based System	179
1. Reset Circuit	179
2. Clock Circuit	179

3. Pull-up Resistors (only if Port 0 is used as I/O Port)	179
4. Demultiplexer	179
11.5 Troubleshooting 8051 based Systems	179
11.6 Program Memory Protection	180
<i>Points to Remember</i>	180
<i>Objective Questions</i>	181
<i>Review Questions with Answers</i>	181
<i>Exercise</i>	182
12. The 8051 Programming in C	183
12.1 Data Types for The 8051 Supported by Cx51 Compiler	184
12.1.1 Native Word Size: char	184
12.1.2 Additional Data Types for the 8051	185
1. Bit	185
2. sfr	185
3. sbit	185
12.1.3 Implementing Infinite Loops in a C Program	188
12.1.4 Bit Addressing in the C Language	191
12.2 Accessing Memory Areas of The 8051	195
12.2.1 Internal RAM (Data Memory)	195
12.2.2 External RAM (Data Memory)	196
12.2.3 Program/Code Memory	196
12.3 Bit Addressable Variables	196
12.4 Interrupt-Service Routines	197
‘using’ attribute	197
12.5 Operators	197
12.6 Data Serialization using Port Pins	201
12.7 Rotate Operations in C	202
12.7.1 Left Rotation	203
12.7.2 Right Rotation	203
12.8 Pointers	204
12.9 Pointers to Absolute Addresses	205
12.10 Time Delays in C	206
12.11 Increasing the Code Efficiency	207
12.11.1 Variable Size	207
12.11.2 Use of Unsigned Data Types	207
12.11.3 Use of Bit Variables	207
12.11.4 Inline Functions	207
12.11.5 Use of Internal RAM	207
12.11.6 Inline Assembly/Hand-Coded Assembly	207
12.11.7 Avoid Standard Library Routines	207
12.11.8 Use of Intrinsic Functions	207
12.12 Performance Comparison between Assembly and C Programs	207
<i>Points to Remember</i>	212
<i>Objective Questions</i>	212
<i>Review Questions with Answers</i>	213
<i>Exercise</i>	214
13. Input/Output Ports	215
13.1 The 8051 Ports	216
13.1.1 Port 1	216
1. Configuring the Port as an Input	217
2. Configuring the Port as an Output	217
13.1.2 Port 0	219
Port 0 as an Address/Data Bus	219
13.1.3 Port 2	220
13.1.4 Port 3	221
13.2 Reading Latch Versus Reading Port Pin	225
Logical Operation with Ports	226
13.3 Port Current Capabilities	227
<i>Points to Remember</i>	227
<i>Objective Questions</i>	227
<i>Review Questions with Answers</i>	228
<i>Exercise</i>	229
14. Timers	230
14.1 Need of Timers	231
14.2 How Does a Timer Operate?	231
14.3 Timers in The 8051	231
14.3.1 TMOD (Timer Mode Control) Register	232
2. C/T	233
3. M1 and M0	234
14.3.2 TCON Register	235
14.4 Timer Circuits as an Interval Timer	236
14.4.1 Timer Mode 0	236
14.4.2 Timer Mode 1	236
1. Operation of the Timer in Mode 1	236
2. Initial Value to be Loaded in Timer Registers	237
3. Square-Wave Generation using Timers	238
4. Timer Mode 0	243
14.4.3 Timer Mode 2	244
Operation of Mode 2	245
14.4.4 Generating Larger Delays	247
14.4.5 Timer Mode 3	250
14.4.6 Reading the Value of a Timer	250
14.5 Timer as an Event Counter	252
Simulation Procedure	253
14.6 Frequency Measurement using Timers	254
<i>Points to Remember</i>	256
<i>Objective Questions</i>	256
<i>Review Questions with Answers</i>	257
<i>Exercise</i>	258
15. Serial Communications	259
15.1 Need for the Serial Communication	260
15.2 Synchronous and Asynchronous Serial Communications	260

15.2.1	Format of Asynchronous Serial Data Frame	261
15.2.2	Rate of Data Transfer	262
15.3	Rs 232 : Serial Data Transmission Standard	262
15.3.1	Hand-shaking Process between DTE and DCE	262
15.3.2	RS232 to TTL Interfacing	263
15.4	UART	264
15.4.1	UART Features	265
15.4.2	SBUF (Serial Data Buffer) Register—One Name—Two Registers	265
15.4.3	SCON—Serial Control Register	265
15.4.3	Serial Port Control (SCON) Register	265
15.5	Modes of Operation	266
15.5.1	Mode 0—8 bit Shift Register Mode	266
	Transmission	266
	Reception	266
	Baud Rate for Mode 0	269
15.5.2	Mode 1—Standard 8-bit UART Mode	269
	Transmission	269
	Reception	270
	Baud Rate for Mode 1	270
	Software Development to Transmit and Receive Data Serially	272
	Simulation Result	274
	Simulation Procedure	275
15.5.3	Mode 2—Multiprocessor Communication	280
	Transmission	281
	Reception	281
	Multiprocessor Communication	281
	Baud Rate for Mode 2	282
15.5.4	Mode 3	282
15.6	Second Serial Port in The DS89C4x0	283
	<i>Points to Remember</i>	285
	<i>Objective Questions</i>	285
	<i>Review Questions with Answers</i>	287
	<i>Exercise</i>	288

16. Interrupts 289

16.1	Need of Interrupts	290
16.1.1	How are Interrupts Useful?	290
16.2	Interrupts in The 8051	290
16.2.1	Reset as a Special Interrupt	292
16.3	Interrupt Handling and Execution	292
16.4	Programming the Interrupts	293
16.4.1	Interrupt Enable (IE) Register	293
16.4.2	Interrupt Priority (IP) Register	294
16.5	Timer Interrupts	296
16.5.1	Programming of Timer Interrupts	296
16.5.2	Simultaneous and Independent use of both the Timers	297
	Finding the Count for 500 Hz Frequency	298
	Finding the Count for 10 KHz Frequency	298

	Analysis of the Program	299
	Simulation Result	299
	Finding the Count for 2 KHz and 4 KHz Frequency	302
	The Count for 100 Hz Frequency	302
16.6	External Interrupts	304
16.6.1	Level-Triggered External Interrupts	304
16.6.2	Transition (Edge) Triggered External Interrupts	304
	Simulation Procedure	305
16.6.3	Pulse Generation using External Interrupt	306
16.6.4	Sampling of Edge-Triggered Interrupts	306
16.7	Serial Port Interrupts	307
	The More Efficient Method	308
	Simulation Procedure	308
	Simulation Result	310
16.8	Interrupt Priorities	312
16.9	Nested and Multiple Interrupts	312
16.10	Blocking Conditions	313
16.11	Interrupt Latency	314
16.12	Generating Interrupts using Instructions	315
16.13	Cautions While Developing Interrupt Service Routines	315
16.14	Dilemma: Use Interrupt or Polling?	315
16.15	Project: Full-Duplex System	316
	Problem Statement	316
	Program Development	317
	Suggested Modification	318

Points to Remember 318

Objective Questions 318

Review Questions with Answers 320

Exercise 320

17. Interfacing Keyboards 322

17.1	Keyboard Design Considerations	323
17.1.1	Mechanical Properties of the Switches	323
17.1.2	Key Debouncing using Hardware	324
17.1.3	Key Debouncing using Software	324
17.2	Keyboard Configurations	324
17.2.1	Simple Keyboard Configuration (Using I/O Pins directly)	324
	Advantages of a Simple Keyboard	324
	Disadvantages of Simple Keyboard	325
	Key-Press Detection and the Code Generation	325
	Algorithm	325
	Program for a Simple Keyboard	325
	Key Identification using the Hardware Technique	327
	Advantage of Hardware Technique	327
17.2.2	Matrix Keyboard Configuration	328
	Key-Code Generation	328
	Key Identification and Key Code Generation using Counters	329
	Key Identification and Key-Code Generation using the Look-Up Table	331

Points to Remember 331
Objective Questions 332
Review Questions with Answers 332
Exercise 332

18. Interfacing Display Devices: LED, Seven Segment Display and LCD 333

- 18.1 Light Emitting Diodes 334
 - Applications of LEDs 336
 - 18.2 Seven-Segment Display 337
 - 18.2.1 Segment Multiplexing within one Seven-Segment Display 337
 - 18.2.2 Digit Multiplexing 337
 - 18.3 Liquid Crystal Display (LCD) 340
 - Advantages of using LCD as a Display Device 340
 - 18.3.1 Pin Description for LCD 340
 - 18.3.2 LCD Commands 341
 - 18.3.3 Initialization of the LCD using the Internal Reset Circuit 341
 - 18.3.4 Software Initialization of the LCD 341
 - 18.3.5 LCD Timing 341
 - 18.3.6 Modes of operation 341
 - 8-bit Mode 344
 - Importance of Monitoring Busy Flag 348
 - 4-bit Mode 350
 - 18.4 Project: Simple Burglar Alarm System 350
 - Program Development 351
 - Suggested Modifications 353
- Points to Remember* 353
Objective Questions 353
Review Questions with Answers 354
Exercise 354

19. Interfacing ADC, DAC and Sensors 355

- 19.1 Analog to Digital Converters 356
 - 19.1.1 Need for Analog-to-Digital Converters 356
 - 19.1.2 Methods of Conversion 356
 - 19.1.3 ADC Parameters 356
 - 19.1.4 Common ADC Chips 357
 - 19.1.5 ADC 0801/02/03/04/05 Chips 358
 - Analog inputs 358
 - Analog Input Voltage Range 358
 - Digital Output 359
 - Clock Source 359
 - 19.1.6 Handshaking Process between the Microcontroller and ADC 0804 Chip 360
 - 19.1.7 ADC 0808/0809 Chips 362
 - 19.1.8 Serial ADC Chips 364
 - Serial ADC Chip MAX1112/MAX1113 364
 - Pin Description of MAX1112/1113 365
 - Control Byte of MAX1112/MAX1113 366
 - Sending Control Byte to MAX 1112/MAX1113 367

Reading Digital Output Data (Result) from the Serial ADC Chip 367
 Common Serial ADC Chips 370

- 19.1.9 On-chip ADCs 371
 - P89LPC768 371
- 19.1.10 Applications of ADCs 373

- 19.2 Digital-to-Analog Converters 373
 - 19.2.1 DAC Parameters 373
 - 19.2.2 Common DAC chips 374
 - 19.2.3 DAC AD557 Chip 374
 - Program to generate Sine Wave using DAC AD557 375
 - Program 376
 - 19.2.4 DAC 0808 Chip 378
 - Operation of DAC0808 378
 - 19.2.5 Applications of DACs 381
 - 19.3 Temperature Sensor: LM35 382
 - 19.3.1 LM35 Specifications 382
 - 19.3.2 Common Temperature Sensors 383
 - 19.3.3 Applications of Temperature Sensors 383
 - 19.4 Infrared (IR) Sensors 384
 - 19.4.1 TSOP 17xx IR Receivers 384
 - 19.4.2 Interfacing of TSOP 17xx with the 8051 385
 - 19.4.3 Applications of IR Sensors 385
- Project: *Temperature Monitoring System* 385
 Project: *Function Generator* 389
Points to Remember 393
Objective Questions 394
Review Questions with Answers 395
Exercise 395

20. Interfacing Relays, Opto-Couplers, Stepper and Dc Motors 396

- 20.1 Relays 397
 - 20.1.1 Relay Operation 397
 - 20.1.2 Relay Driver Circuits and Interfacing 397
 - Driver Circuit Operation 398
 - Advantages of using a Relay 399
 - Drawbacks of Electromechanical Relays 399
 - 20.1.3 Parameters of Relays 399
 - Input Coil Side (Low Power, Controlling Side) 399
 - Output Switch Side (High Power, Load Side) 399
- 20.2 Opto-Coupler 400
 - 20.2.1 Opto-Coupler Operation 400
 - 20.2.2 Applications of Opto-Couplers 401
- 20.3 Stepper Motors 401
 - 20.3.1 Permanent-Magnet Stepper Motors 401
 - 20.3.2 Unipolar and Bipolar Stepper Motors 402
 - 20.3.3 Direction and Speed Control 404
 - 20.3.4 Interfacing with the 8051 404
 - Circuit Operation 405
 - 20.3.5 Rotation of Motor for Specified Angle 408

20.3.6	Applications of Stepper Motors	409
20.4	DC Motors	409
20.4.1	Analog Speed Control	409
20.4.2	Digital Speed Control	410
20.4.3	Direction Control	410
20.4.4	Pulse-Width Modulation (PWM)	412
20.4.5	DC Motor Driver Circuits	412
20.4.6	Interfacing DC Motors with the 8051	414
	Project: <i>DC Motor-Speed-Control System</i>	416
	Project: <i>Automatic Street Light Control System</i>	420
	<i>Points to Remember</i>	421
	<i>Objective Questions</i>	422
	<i>Review Questions with Answers</i>	422
	<i>Exercise</i>	423

21. Interfacing External Memory and Real-Time Clock 424

21.1	Memory Interfacing and its Need	425
21.2	Memory Chips	425
21.2.1	Address Signals	425
21.2.2	Data Signals	426
21.2.3	Control Signals	426
21.3	Semiconductor Memory Devices	426
21.3.1	Volatile Memory	426
	SRAM: Static RAM	427
	DRAM: Dynamic RAM	427
21.3.2	Nonvolatile Memory	427
	ROM	427
	PROM: Programmable ROM	428
	EPROM: Erasable Programmable ROM	428
	EEPROM: Electrically Erasable and Programmable ROM	428
	Flash	428
	NVRAM: Nonvolatile RAM	428
21.4	Memory Map and Address Decoding	429
21.4.1	Signals used in Memory Interfacing	429
21.4.2	The 8051 and the Corresponding Memory Chip Signals	429
	Address Lines ($A_{15}-A_0$)	429
	Data Lines (D_7-D_0)	429
21.4.3	Address Decoder using Logic Gates	430
	Memory Read and Write Operations	431
21.4.4	Address Decoder using Decoder Chip	431
21.5	Program/Code Memory Interfacing	433
21.6	Data/RAM Memory Interfacing	435
21.7	Data Memory Using ROM	437
21.8	ROM as Data as well as Program Memory	437
21.9	RAM as Data as well as Program Memory	438
21.10	On-Chip EEPROM Programming in AT89S8253	438
	Steps to Write (Program) a Byte in Data EEPROM	440
	Steps to Write a Page (Maximum 32 Bytes) in Data EEPROM	441
	Steps to Read a Byte(s) from Data EEPROM	441
21.11	Real-Time Clock	441

21.11.1	DS12887: Real-time Clock Chip	443
	Main Features of the DS12887	443
21.11.2	Address Map	443
21.11.3	Interfacing DS12887 with the 8051	443
21.11.4	Programming the DS12887	444
	Oscillator Control Bits	444
	Setting the Time and Date	445
	Reading the Time and Date	447
21.11.5	Square Wave Output	448
21.11.6	Alarms	448
21.11.7	Periodic Interrupts	450
21.11.8	Update Cycles	450
21.11.9	Interrupt Sources	450

Points to Remember 450

Objective Questions 450

Review Questions with Answers 451

Exercise 452

22. I2C and SPI Protocols 453

22.1	Inter Integrated Circuit (IIC or I ² C)	454
	I2C Bus Features	455
22.2	I2C Bus Hardware Configuration	454
22.2.1	V_{CC} or V_{DD}	455
22.2.2	I2C Devices	456
22.3	I2C Protocol	455
22.3.1	START and STOP Conditions	456
22.3.2	Data Validity	457
22.3.3	Data Transfer Operations on I2C Bus	457
22.3.4	Write Operation	457
22.3.5	Read Operation	459
22.3.6	Arbitration and the Clock Synchronization	460
22.3.7	Clock Stretching	460
22.3.8	Burst Read/Write Modes	461
22.4	Driving The I2C Bus	460
22.4.1	I2C Interface Module of P89C66x Microcontrollers	462
	S1CON: Serial 1 Control (I2C control) Register	462
	S1STA: Serial 1 Status (I2C Status) Register	463
	S1DAT: Serial 1 Data (I2C Data) Register	463
	S1ADR: Serial 1 Address (I2C Address) Register	463
22.4.2	Programming I2C Interface of P89C66x	463
	Using I2C Interface as a Master	463
	Initialization of I2C Module	463
	Generate START Condition	463
	Transmit Data	464
	Read Data	464
	Generate a STOP Condition	464
22.4.3	Interfacing PCF8594C-2 Serial EEPROM	466
	Addressing of PCF8594C-2	467
	Write Operation	467
	Read Operation	467
	24xxx and AT24Cxxx Serial	

EEPROMs	467	23.6.5	ADC and DAC	488
Using I2C Device as a Slave	470	23.6.6	Real-Time Clock (RTC)	488
Initialization of I2C Module	470	23.6.7	Other Peripherals and Features	488
Monitor the Bus Condition	470		Multiple DPTRs	489
Transmit Data	470		Four Levels of Interrupt Priorities	489
Read Data	470		In-System Programming (ISP)	489
22.5 I2C Devices	469		In-Application Programming (IAP)	489
22.6 Serial Peripheral Interface	470	23.7	MCS 151/251 Microcontrollers	488
22.6.1 SPI Operation	471	23.8	MCS 96 Microcontrollers	489
22.6.2 Clock Polarity and Phase in SPI Device	472	23.9	AVR Microcontrollers	490
22.6.3 SPI Bus Configurations	473	23.9.1	AVR ATmega Family	493
22.7 AT89S825x	472	23.9.2	Programming Model of ATmega16 Family of Microcontrollers	493
22.7.1 SPI Interface Module of AT89S825x Microcontrollers	474		General-Purpose Registers: R0–R31	493
SPCR: SPI Control Register	474		STATUS Register	494
SS Pin	474		Program Counter	495
SPSR—SPI Status Register	474		Stack Pointer	495
SPDR – SPI Data Register	475		Data Memory	495
22.7.2 Interfacing MAX512/13 with SPI Bus	477		Data EEPROM	495
Serial-Input Data Format and Control Codes for MAX512/13	478		Program Memory	496
22.7.3 Interfacing MAX512/13 with AT89S8253	478		I/O Ports and Peripherals	496
Program Development	479	23.10	PIC Microcontrollers	495
22.8 SPI Devices	479	23.10.1	PIC18 Family	497
22.9 Comparison between I2C and SPI Protocols	479	23.10.2	Programming Model of PIC18 Family of Microcontrollers	498
<i>Points to Remember</i>	479		Working Register (WREG)	498
<i>Objective Questions</i>	480		Bank Select Register (BSR)	499
<i>Review Questions with Answers</i>	481		File Select Registers (FSR)	499
<i>Exercise</i>	482		STATUS Register	499
			Table Pointer	500
23. The 8051 Variants, AVR and PIC Microcontrollers	483		Program Counter	500
23.1 The 8051 Enhancements	484		Stack and Stack Pointer	500
23.1.1 Additional 128 Bytes of On-Chip RAM	485		Special Function Registers (SFRs)	500
23.1.2 Timer 2	485		Data Memory	500
23.1.3 Maximum Clock Speed	485		Program Memory	501
Clocks/Machine Cycle	485		Data EEPROM Memory	501
23.1.4 Program Memory Identification	485		I/O Ports	502
23.2 8051 Variants from NXP (Philips)	485		<i>Points to Remember</i>	501
23.3 8051 Variants from Atmel Corporation	485		<i>Objective Questions</i>	501
23.4 8051 Variants from Dallas Semiconductor	485		<i>Review Questions with Answers</i>	502
23.5 8051 Variants from Silicon Laboratories	486		<i>Exercise</i>	502
23.6 Common On-Chip Peripherals	486		Appendix A: <i>The 8051 Instruction Set Summary</i>	503
23.6.1 Watchdog Timer	487		Appendix B: <i>Using Keil μVision 4.0 IDE</i>	525
23.6.2 Controller Area Network (CAN)	488		Appendix C: <i>Instructions Arranged Functionally</i>	540
23.6.3 Analog Comparator	488		Appendix D: <i>ASCII Codes</i>	549
23.6.4 Pulse-width Modulator	488		Appendix E: <i>Special Function Registers Quick View</i>	551
			Index	555

Preface

In the past few decades, microcontrollers have changed the way we live, entering almost all aspects of our life. Their production counts are in billions per year. Due to the massive applications of microcontrollers, there is enough space for 8-bit microcontrollers for small- and medium-scale embedded systems in the coming future. Embedded software is used in almost every electronic device today. Therefore, there is a substantial need of skillful programmers and system designers. Moreover, the availability of sufficient hardware resources in recent variants of microcontrollers has led to universal use of high-level languages such as C (except very time-critical applications) in the development of embedded systems.

Target Audience

This book is specifically written for an introductory (first) course in the subject, and the contents are class tested to ensure that the treatment is logical and easy to understand for the fresher.

This book can be used by students as a text/reference book for either one- or two-semester courses at the undergraduate level, i.e. in Bachelor of Engineering (Electronics & Communication, Electronics, Computer Science, Information Technology, Instrumentation & Control, Mechatronics, Electrical Engineering, etc.). It can also serve as a reference book for Bachelor of Science or Master of Science in the field of Electronics, Diploma courses, technical training institutes and embedded-system designers.

The text assumes that the readers are familiar with concepts and terminology of digital systems.

Rationale behind Writing this Book

During my interactions with students over several years, undertaking the subject based on the 8051 microcontroller, I found they face several difficulties in studying the subject because of the ***lack of a standalone book*** with easy-to-understand and reader-friendly language, in-depth coverage of topics with balanced treatment of fundamental concepts and practical aspects with applications, proper organization and flow of content. Even I have faced similar difficulties while teaching the subject for the first few times. Keeping in mind these issues, I was inspired to write a book that could fulfill the needs of students and could serve as a standalone reference.

The objective of the book is to introduce fundamental hardware, software and architectural aspects of microcontroller-based embedded systems in an elementary and integrated manner and to provide a strong foundation for the development of expertise in designing such systems.

About the Book

This book covers topics the author feels every embedded-system designer must know. The 8051 microcontroller is chosen as the subject as it is the most popular 8-bit microcontroller due to its low cost, easy availability of tools and support, multiple vendors and wide variety of variants, number of companies licensing the core with new peripherals for continuous improvement of their products, the large numbers of 8051 aware engineers (both hardware and software) and reusability of existing 8051 software in the public domain.

The following aspects of embedded-systems design are discussed in the book using the 8051 microcontroller as an illustration.

• Architecture

- Architectural block diagram and Programming model of the 8051
- Timing diagrams and instruction execution

• Hardware features and resources

- I/O ports, Timers/Counters, Serial data transfer protocols: UART, I2C and SPI, Interrupts
- System design and Troubleshooting, Introduction to common peripherals and features: CAN, Watchdog timers, PWM, Analog comparators, Multiple DPTRs, ISP and IAP, Comparison and Introduction of MCS 51, MCS 96, MCS 151, MCS 251, AVR ATmega and PIC 18 microcontroller families

• Programming

- Instruction set, Logic/program development steps, Assembly and C language programming with powerful documentation, Debugging and testing the programs with simulation steps and snapshots, Tools for software development

• Real-world interfacing

- Keyboards, Memory (data as well as code memory), EEPROM programming, Display devices: LED, LCD, Seven segment displays, ADCs(On-chip and Off-chip) and DACs (Serial as well as parallel), Real-Time Clock, Stepper motors, dc motors, LM35 temperature sensors, IR sensors, Relays and Opto-couplers, Projects

The presentation of the introduction and background of all concepts is general in nature, and detailed discussion is based on the Intel 8051 microcontrollers and their variants. This approach allows the reader to migrate easily to other microcontroller architectures.

The potential reader can easily understand the importance and need of the book by observing the organization of the topics in the table of contents. For beginners, the theoretical concepts given in the book establish a strong foundation necessary for development of microcontroller-based embedded systems. For experienced readers/professionals working on the projects, it provides detailed coverage of topics and may serve as a practical and reference guide. The book also helps teachers arrange the flow of content best suited for classroom teaching, discussions and presentations.

The book covers a wide variety of latest variants of the 8051 microcontroller that can compete with other microcontroller architectures in the market, a tutorial on the latest software-development tool—IDE (Keil μ Vision 4.0). It also has complete chapters on advanced serial data-transfer protocols like I2C and SPI, and timing and execution of all types of instructions with the help of data-flow diagrams, introduction to MCS 96, MCS 151, MCS 251, PIC18 and the AVR ATmega family of microcontrollers.

This book can be used to establish the strong background for taking advanced subjects like embedded system design, embedded computer architecture, RTOS and microcontroller architecture.

Chapter Organization

The major theme of the book is ‘logical sequencing of the chapters and their topics with proper organization and flow of content to enhance understanding of the entire subject’. The major focus is on concise, to-the-point discussion of topics with **clarity and simplicity**.

The book is divided logically into three parts.

Part 1: Microcontroller Architecture, Programming and Development Tools (Chapters 1 to 12)

Chapter 1 covers the basics of computer systems and microcontrollers. The applications, classification and criterion for selection of microcontrollers for specific applications is discussed. The features of the 8051 family of microcontrollers and comparison with other microcontroller families are given in brief. **Chapter 2** introduces architectural block diagram and programming model of the 8051. **Chapter 3** is dedicated to the tools and program-development process. **Chapters 4 to 8** cover instruction set of the 8051, programming concepts and show how to use the instructions to develop simple programs.

Chapter 9 is devoted to programming examples for array processing, 16-bit arithmetic and code conversions. **Chapter 10** explains timing diagrams and execution of all types of instructions with the help of data-flow diagrams. **Chapter 11** presents the pin diagram of the 8051 and shows how standalone systems using the 8051 can be designed. **Chapter 12** focuses on programming in C language. It shows how the hardware features of the 8051 can be used and controlled by a high-level language.

Part 2: On-chip Peripherals (Chapter 13 to 16)

Chapter 13 discusses in detail the port structure of the 8051. It shows how the ports of 8051 can be used to interface the microcontroller with the external world. **Chapter 14** discusses the need and uses of timers. It covers adequate details of different modes of the timer operation, programming the timers as interval timers as well as event counters. It contains a wide variety of examples in assembly as well as C language related to uses of timers and counters. **Chapter 15** is devoted to serial communications. It discusses different types of communications and need of serial communications. The RS232

standard is introduced, and all modes of UART with programming examples and applications are given in detail. **Chapter 16** discusses the interrupts in detail. The applications and programming of internal and external interrupts are given with sufficient details. Advanced concepts like interrupt priorities, nested interrupts and interrupt latency are introduced.

Part 3: Real-world Interfacing (Chapters 17 to 23)

Chapter 17 covers issues related to designs of keyboards. The major focus is on programming, designing and interfacing different types of keyboards with the 8051. **Chapter 18** is dedicated to display devices like LEDs, 7-segment LEDs and LCDs. It shows methods to program and interface these devices with the 8051. **Chapter 19** focuses on data converters and sensors, i.e. analog-to-digital and digital-to-analog converters, temperature and infrared sensors. The features, programming and interfacing various on-chip and off-chip ADCs, DACs and sensors are discussed in detail. **Chapter 20** discusses the features, design, operation, programming and interfacing circuits of relays, opto-couplers, DC motors and stepper motors. **Chapter 21** is about interfacing external memory to the 8051-based systems. It introduces the types of memories, signals of memory chips and address-decoding methods. The method of on-chip EEPROM programming is also presented. The interfacing, programming and applications of Real-Time Clock chip DS12887 is given in detail. **Chapter 22** covers in detail the advanced serial data-transfer protocols: I2C and SPI. It shows, with numerous examples, how this protocols can be used in real-life applications. **Chapter 23** introduces and compares the features of different variants of the 8051 microcontrollers from various chip manufacturers. It also covers MCS 96, MCS 151, MCS251, PIC18 and AVR ATmega family of microcontrollers.

All theoretical concepts are explained with proper examples and illustrations wherever necessary. Large numbers of programming examples are given, which gives a better insight into the theoretical material and makes the book application-oriented. Multiple examples for the same concept help in clarifying any doubt regarding the concept.

The examples in the book cover adequate details of all aspects of programming and real-world interfacing like logic/program development steps, optimization with respect to execution speed and memory requirements, powerful comments to help understand, upgrade or modify programs, simulation steps, snapshots of outputs, debugging and troubleshooting techniques, and complete hardware interfacing diagrams. The book includes simple projects; each project includes the problem statement, complete schematic diagram, logic/program development steps, assembly and/or C program and suggested modifications. These projects show how the 8051 can be used to in real-life applications.

Each chapter begins with *Learning Objectives* and *Key Terms* that provide an idea about specific outcomes from the chapter. Pictorial illustrations of a majority of fundamental theoretical concepts aid in thorough understanding of the subject.

Objective-type Questions (MCQ), *Review Questions with Answers* and *Points to Remember* at the end of the chapters are sufficient to enforce the application of concepts understood in the chapter and will help students prepare themselves for self-test as well as examinations. *Think Boxes* in the text are given at suitable places to highlight miscellaneous concepts and to avoid confusions where students might stumble or get confused.

How to use the Book

Once the theoretical concepts are understood after reading the topics, the programming examples can be tested in the IDE-µVision 4.0 to have better insight of the topic. Appendix B explains how to use µVision 4.0 to develop, simulate and debug 8051 programs in assembly as well as C language. To support and ease understanding, a stepwise explanation along with screenshots of µVision 4.0 IDE windows is given for sample programs.

Salient Features

- ◆ Simple and easy-to-understand language supported with self-explanatory diagrams
- ◆ Logical sequencing of topics, concise and to-the-point discussion
- ◆ Step by step approach for the software development
- ◆ Pictorial explanation of majority of concepts followed by examples for ease of understanding
- ◆ Latest advancements to the field like I2C, SPI, etc., not present in any book
- ◆ Simulation methods and snapshots of the output for some key examples
- ◆ Programming examples in assembly and C languages; all instructions explained through use of examples

- ◆ Timing and data-flow diagrams for instruction execution
- ◆ Advanced and complex topics like interrupt handling, interrupt latency, lookup tables, timing analysis, stack operations, multiprocessor communications, 8051 enhancements and variants, internal port structure covered with clarity
- ◆ Coverage of many variants, peripheral devices, PIC and AVR microcontrollers
- ◆ Tutorial of Keil μ Vision4.0 Integrated development environment
- ◆ 6 projects to help students get hands-on experience and improve their designing skills
- ◆ Excellent pedagogy:
 - *Learning Objectives* and *Key Terms* at the beginning of each chapter
 - *Points to Remember* at the end of each chapter
 - Discussion Questions within the topics: 25
 - Review Questions with answers: 310
 - Exercise Questions: 410
 - Programming Examples (Assembly and C): 325
 - Objective Questions at the end of each chapter: 301
 - Think Boxes with Answers: 95
 - Illustrations (Figures and Tables): 350

Online Learning Center

The book is supplemented with separate online resources for instructors and students, accessible at <http://www.mhhe.com/patel/mbes>

Online Resources for Instructors

- ◆ Complete Solution Manual of the book
- ◆ Chapterwise PowerPoint slides
- ◆ Additional material on advanced microcontrollers

Online Resources for Students

- ◆ *Lab Manual*: A complete Lab Manual containing 14 laboratories with sample references from the book, sample programs followed by laboratory exercises to reinforce the concepts
- ◆ Chapterwise Objectives, Key Terms and Points to Remember
- ◆ *Projects given in the book*: Each project includes the problem statement, complete schematic diagram, program development, assembly and/or C programs and suggested modifications
- ◆ Question papers of different universities with solutions
- ◆ *Chapterwise interfacing diagrams*
- ◆ Complete designs (Schematic diagram and PCB layout) of the 8051 based hardware boards
- ◆ Additional question bank

Acknowledgements

I am highly indebted to Dr. Nikhil Kothari, Head, Electronics and Communication Department, Faculty of Technology, who inspired me to write this book and taught me a majority of the concepts covered in the book. I would like to express my sincere thanks to Dr. H M Desai, Vice Chancellor, Dharmsinh Desai University, and Prof. D G Panchal, Dean, Faculty of Technology, Dharmsinh Desai University, for providing a creative and challenging atmosphere in the university campus.

I am grateful to my colleagues and friends—Prof. V A Vohra, Prof. D K Rabari, Prof. B P Patel, Prof. R K Dana, Prof. B B Patel, Prof. S S Thavalapill, Dr. V M Thumar, Prof. P D Dalal and Prof. H D Patel—for their reviews, suggestions and support during the development of the manuscript. I am thankful to my students for their valuable feedback/reviews of the manuscript and their help in preparing the material of the book. I am thankful to Mr. Nitin Paranjape (MD, Edutech

Systems, Vadodara) for providing me the details of hardware boards and tools. I express my acknowledgement to the entire team at McGraw-Hill Education India, especially, Mr Sourabh Maheshwari and Mr Piyaray Pandita for their support and guidance.

I thank Intel Corporation, NXP Semiconductors and Atmel Corporation for generously allowing me to use the information from their product datasheets. My special thanks to Keil Corporation for allowing me to use snapshots of their IDE μ Vision 4.0.

I am deeply indebted to my parents; wife Dr. Devnagi, daughter Ragvi, and family for their constant motivation and understanding.

The following reviewers also deserve a special mention for sending me their feedback and suggestions.

K Venkata Reddy	<i>Jawaharlal Nehru Technological University (JNTU), Kakinada, Andhra Pradesh</i>
Padmavathi P	<i>Malla Reddy Institute of Technology and Sciences, Hyderabad, Andhra Pradesh</i>
B Bhavani	<i>Maturi Venkata Subba Rao (MVSR) Engineering College, Hyderabad, Andhra Pradesh</i>
V Seetha Lakshmi	<i>Sri Sakthi Institute of Engineering and Technology, Coimbatore, Tamil Nadu</i>
K R Anil Kumar	<i>NSS College of Engineering, Palakkad, Kerala</i>
Suresh Kumar	<i>Vellamal Engineering College, Surapet, Tamil Nadu</i>
C A Ghuge	<i>PES Modern College of Engineering, Pune, Maharashtra</i>
Rachit K Dana	<i>Dharmsinh Desai University, Nadiad, Gujarat</i>
Anil Kumar Sharma	<i>Abacus Institute of Engineering and Management, Hooghly, West Bengal</i>
Pinaki R Ghosh	<i>Adamas Institute of Technology, Kolkata, West Bengal</i>
Santanu Chattopadhyay	<i>Indian Institute of Technology (IIT) Kharagpur, West Bengal</i>
Chanchala Kumari	<i>National Institute of Technology (NIT) Jamshedpur, Jharkhand</i>
P K Mukherjee	<i>Indian Institute of Technology-Banaras Hindu University (IIT-BHU), Varanasi, Uttar Pradesh</i>
Praveen Malik	<i>Shastri Nagar, Meerut, Uttar Pradesh</i>

Feedback Request

I will be grateful to the readers if they have suggestions (feedback/criticism/comments) and can point out errors. These can be sent to manishpatel_79@yahoo.com

Manish K Patel

Publisher's Note

McGraw Hill Education (India) invites suggestions and comments from you, all of which can be sent to tmh.csefeedback@gmail.com (kindly mention the title and author name in the subject line).

Piracy-related issues may also be reported.

Introduction to Microcontrollers

Objectives

To discuss and Introduce:

- ◆ Basic structure, organization and functions of a computer system along with common terminology of computer literature
- ◆ Features and comparison of microprocessors and microcontrollers
- ◆ Microcontroller classification with respect to core, memory and instruction set architecture
- ◆ Criterion for selection of microcontroller with respect to target product
- ◆ Applications of microcontrollers in various fields
- ◆ Features, overview and advantages of using the 8051 family microcontrollers
- ◆ Variants and enhancements of the 8051
- ◆ Concept of embedded systems and their characteristics

Key Terms

- | | | |
|-------------------------------|-------------------------------|-------------------------------------|
| • 8051 Family | • Control Bus | • Microcomputer |
| • Address Bus | • Data Bus | • Microcontroller |
| • ALU | • Embedded System | • Microprocessor |
| • Arithmetic/Logic Operations | • Input/Output (I/O) Unit | • Peripherals |
| • Boolean Processor | • Instructions | • Program/Data Memory |
| • Central Processing Unit | • Memory | • Von Neumann/Harvard Architectures |
| • CISC/RISC | • Microcoded/Hardwired Design | • Word Length |

A computer is a digital device which understands and operates on binary digits 0 and 1, known as *bits (binary digits)*. The binary digits are processed and stored as voltage levels in the circuits. The computer system needs bit patterns of 0s and 1s to perform any operation. These binary patterns are called *binary instructions* which are recognized and processed (or executed) by a computer to accomplish a task. The designer of the computer decides and implements these bit patterns based on the number and types of operations a computer is required to perform. The most common operations are storing and retrieving binary numbers, arithmetic and logical operations on binary numbers.

1.1 | COMPUTER SYSTEM

A digital computer system typically consists of four major components: the Central Processing Unit (CPU), Memory, Input/Output (I/O) Unit and System Bus. The simplified arrangement of these components in a computer system is shown in Figure 1.1.

1.1.1 Central Processing Unit

The brain of a computer is the central processing unit. It consists of a group of circuits that determine the operations that the computer can perform. The CPU controls the flow of information among the components of the computer. It also processes the data by performing digital operations on them. The block diagram of the CPU is shown in Figure 1.2.

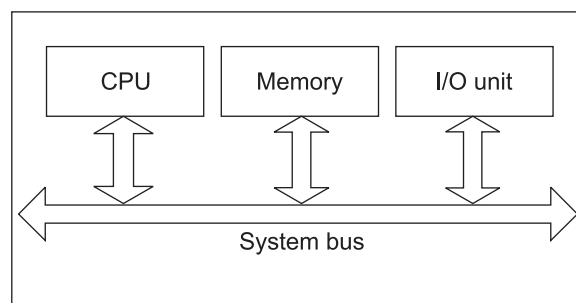


Fig. 1.1 Basic components of a computer system

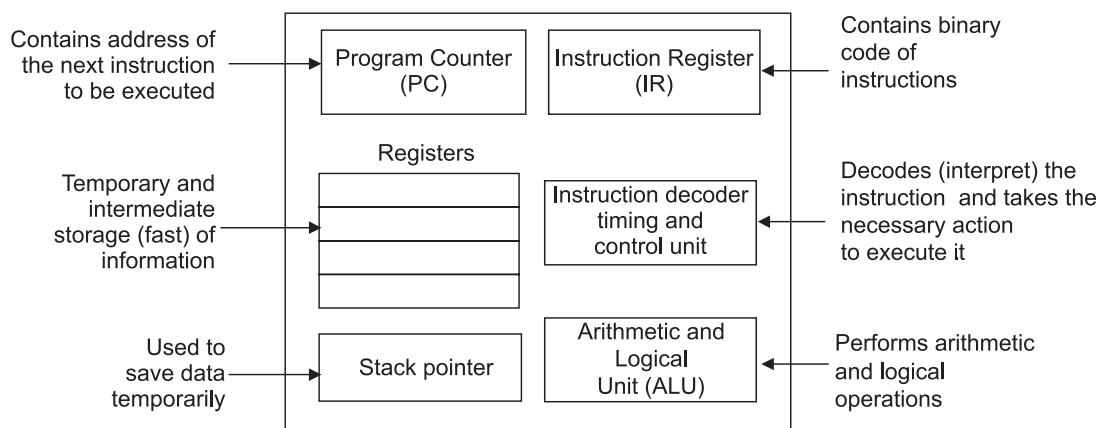


Fig. 1.2 General block diagram of a CPU (microprocessor)

CPU Components

All major components as shown in the block diagram of the CPU are discussed briefly in the following section.

(a) Arithmetic and Logic Unit (ALU) The heart of a CPU is the arithmetic and logic unit which is used to perform all arithmetic and logical operations. Addition, subtraction, multiplication and division are arithmetic operations while AND, OR, NOT, EXCLUSIVE-OR and shifting are logical operations.

The operation to be performed by ALU is decided by control signals generated by instruction decoder and timing unit as per the instruction being executed. Two (or more) inputs are usually given to ALU and it will generate the result of the operation. The ALU also updates the status register to indicate the nature of the result.

(b) Registers The registers are used for holding source data and results of operations temporarily. It consist of general-purpose registers and some dedicated registers to perform specific tasks. For example, accumulator register, which is required for most (arithmetic and logical) operations. Since external memory access is slower than access to these registers (because they are part of the CPU), it is preferred to use the registers when several operations must be performed on a set of data. The registers are also referred as *working registers of CPU*.

(c) Instruction Register (IR) The instruction register holds the binary (machine) code of current instruction while it is being decoded and executed*.

(d) Program Counter (PC) The program counter holds the address of the next instruction to be executed, i.e. it points to the instruction that is to be executed next. As the CPU reads the instructions from the memory, the PC is incremented automatically to point to next instruction. The reading of an instruction (or data) from memory is also referred as fetching an instruction. When the execution of current instruction is completed, the address in the PC is placed on the address bus and in response to that the memory places binary code for the next instruction on the data bus, which is then moved in to instruction register. The length of an instruction (in bytes) will be determined when it is decoded and the PC is incremented such that it will point to the next instruction.*

(e) Stack Pointer (SP) The stack is a portion of the memory used by CPU to save register contents and return addresses temporarily for subroutine and interrupt service routine calls. The data is stored in to stack using PUSH instructions and retrieved using POP instructions.

The address of the stack location which was accessed last is kept in the stack pointer.

(f) Instruction Decoder and Control Unit The instruction decoder interprets the instruction present in the instruction register and control unit determines the sequence of operations that should be performed to complete the task specified by an instruction. Control unit also provides timing to all operations. The control unit provides the inputs for the ALU (either from registers or memory), decides the operation to be performed, and makes sure that the result is written to the proper location (register or memory).

1.1.2 Memory

The memory is used to store data and binary instructions. It is normally organized as several modules (chips), where each module contains several memory locations. Each location may contain part or all of the data or instruction. CPU reads (fetches) the instructions from the memory and performs operations (indicated by instructions) on data.

A typical memory module is shown in Figure 1.3. It consists of N memory locations of equal length (usually bytes). Each memory location is assigned unique address (0 to $N-1$).

All memory devices (chips) have common set of input and output signals, like address, data and control signals. Address inputs (address lines) are used to identify one memory location out of N locations. They are designated as A_0 to A_N , where N is always one less than total address lines. The number of address lines determines capacity of the memory devices (In fact, the number of memory locations inside the memory chip will decide how many address lines are required). For example, 10 address lines on a memory chip indicate that there are $2^{10} = 1024$ memory locations. The data lines transfer data to or from memory devices. The control signals are used to enable memory device and to control nature of operation like read or write.

Types of memories and their associated circuits are discussed in detail in Chapter 21.

1.1.3 I/O Unit

Input/output units are used to communicate with the external world. The input unit (devices) sends information to the computer system. Output devices send information from computer system to the external world. Keyboard, mouse, switches, A/D converters are all input devices while video screen, LED, LCD, speaker, D/A converters are all output devices.

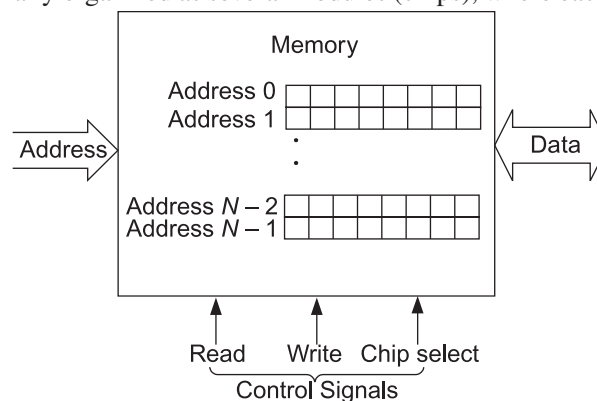


Fig. 1.3 Generalized memory module

* Nowadays, CPUs are available with pipelined architecture where contents of IR changes while execution of current instruction and PC points to next or next to next instruction or even further.

I/O Port, I/O Device and I/O Interfacing Circuits These three terms are used widely in computer hardware literature and are used with blurred distinction between them. Therefore, we need to define them clearly to avoid any confusion.

(a) I/O Port The hardware in a computer that allows information transfer between external world and computer is called I/O port.

These I/O ports are usually eight bits (or multiple of it) wide and thus referred to as a BYTE wide port, i.e. byte wide input port, byte wide output port.

(b) I/O Device (Peripherals) The device that gives information to computer is called input device. For example, keyboard, mouse, joystick, microphone, A/D converters are all input devices.

The device that receives information from computer is called output device. For example, LED, LCD, monitor, printer, speaker, D/A converter are output devices. Memory is both input as well as output device.

(c) I/O Interfacing Circuits The circuits that are used to interconnect (interface) I/O devices with a computer or I/O ports are called I/O interfacing circuits. For example, buffers, latches and voltage level converters are all interfacing circuits. Some I/O devices may also have inbuilt interfacing circuits. The voltage converters are required because different devices may require different operating voltage levels.

THINK BOX 1.1



What are the common input and output devices available in a personal computer?

Keyboard, mouse, CD reader, microphone and joystick are common input devices. LEDs (on the keyboard), monitor (display device), speaker and printer are common output devices. Floppy drive, CD-RW (reader/writer) are input as well as output devices.

1.1.4 System Bus

A group of wires known as a *bus* interconnects the three components of computer systems described above. Physically it is group of 8, 16 or more wires. The system bus provides communication path between CPU, memory and I/O. Three types of buses are required for communications between these three blocks: the address bus, the data bus and the control bus.

Address bus is group of wires used by CPU to identify specific memory location within a memory chip (also to identify specific memory chip out of many chips present in a computer system) and to identify I/O devices as well. Each memory location or I/O device have a unique address, therefore to access them CPU places their address on to the address bus. Since addresses are always generated and placed by CPU, address bus is unidirectional. Size of the address varies across the systems, usually it is 16 bits (wires), 20 bits or 32 bits wide. The size of the address bus (address lines) determines the maximum amount of locations that can be addressed uniquely, i.e. maximum amount of memory that can be present in a system (more commonly referred as address space). If an address is N bits wide then there are 2^N different addresses (0 to $2^N - 1$), hence address space is of 2^N bytes*. For example, a 16-bit address bus can address $2^{16} = 65536$ bytes of memory.

Data bus transfers data or instructions between CPU and memory or I/O devices. It is bidirectional because data can be transferred in both directions, i.e. from CPU to memory (or output devices) or from memory or input devices to CPU. Usually, it is 8 or 16 bits wide for low end computers and 32 or 64 bits wide for high end computers. Advantage of wider data bus is higher rate of data transfer.

Control bus is used to enable memory and I/O devices to perform read or write operations. It regulates all activities on the bus and specifies timing and direction of the data transfer. Read (RD), write (WR) and memory/I/O (M/I/O) are most common control signals.

Along with these four major components, interfacing circuits, as discussed earlier, are required to interconnect these components. Interfacing circuits coordinates signals on the bus which are generated by various components of the system. Memory interfacing circuit may typically contain a logic circuit to decode the address of a memory location. Buffers and latches are most common devices used between system bus and memory or I/O devices. A detailed block diagram of computer including different buses and interfacing circuits is shown in Figure 1.4.

* A memory location usually contains 8 bits—a byte.

1.1.5 How does the CPU Read Data from the Memory Chip?

To read instruction or data from memory, the CPU places address of the desired memory location on the address bus. Next, it sends the control signal to enable the memory chip. Finally, memory chip places contents of addressed memory location on to data bus, and CPU reads content of the data bus by generating read control signal. For operations like memory write, I/O read, I/O write same sequence of steps will be taken except different control signals are generated for each operation. Thus, all buses are used in coordination to perform any task of data transfer.

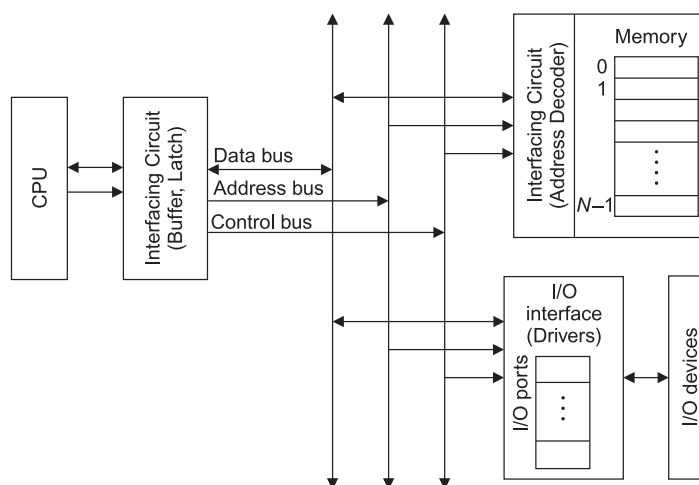


Fig. 1.4 Computer block diagram with buses and interfacing circuits

1.1.6 What Can a Computer Do?

A computer performs broadly only two basic types of operations.

1. Data Movement Moving data from one point to another within a system, this includes moving data within circuits inside CPU, storage and retrieval of data to/from memory and data movement to/from I/O devices.

2. Performing Binary Operations Computers perform mainly two types of binary operations. First, logical operations such as AND, OR, NOT, EXCLUSIVE OR, Shifting. These operations provides decision making and control capabilities if used properly. Second, arithmetic operations such as addition, subtraction, multiplication, division, increment and decrement. Any complex operation is realized by sequence of small operations mentioned above.

It is a general misconception that computer spends its major time and efforts in performing arithmetic and logical operations on data. Actually, it spends little time in these operations. Major time is spent in order to locate the desired data items and in moving them within the system, i.e. from memory (or I/O devices) to CPU and vice versa.

1.1.7 How does Computer Execute Program Instructions?

The process of instruction execution is divided into three cycles.

- 1. Instruction Fetch:** Instruction fetch means reading instruction from memory. During this cycle, CPU sends address of the instruction to a memory through address bus. The memory responds by sending instruction byte (or word) to the CPU, where it is held in instruction register.
- 2. Instruction Decode:** Decoding means interpreting the instruction and to determine sequence of actions that should be taken to perform the operation specified by an instruction.
- 3. Instruction Execute:** In execution cycle, the CPU receives input data either from memory or registers, the result is calculated and finally it is stored back into memory or register as specified in the instruction. All these operations are controlled by control signals generated by a decoder in a predetermined sequence.

Thus, the above three cycles represent overall processing required for a single instruction which is usually referred as instruction cycle. All instructions in a program are executed sequentially by repeating same three cycles. In modern computers, multiple instructions can be fetched, decoded and executed simultaneously.

1.2 | MICROPROCESSOR, MICROCOMPUTER AND MICROCONTROLLER

The definitions and brief introduction of these three terms is given in the following section. The term “micro” in the above three terms means small in size, smaller processing time, but it does not mean small processing power.

1.2.1 Microprocessor

The microprocessor is a central processing unit (CPU) built into a single semiconductor chip. The structure of microprocessor is same as CPU discussed in Section 1.1.1 (central processing unit) and as shown in Figure 1.2. The term CPU was used in early days of computers when CPU was made using discrete components. Microprocessors are

general-purpose and programmable devices, suitable for many applications like word processing, computing, audio/video/image processing, gaming, database management, monitoring and control of machines. They can be made to perform the different tasks as per user's choice which makes possible to design a system with a great flexibility. It is possible to configure a microprocessor based system as a large system or a small system by adding suitable peripherals.

The microprocessor alone is not sufficient to make a functional system. Additional peripherals and circuitry like memory, input/output devices, decoders, drivers, etc., are required to be connected with it to make a functional system.

The microprocessor is often referred as processor, CPU or Microprocessor Unit (MPU) and these terms are used interchangeably.

1.2.2 Microcomputer

The microcomputer is a small computer built using a microprocessor as a central element. It includes all necessary components required for an application. The I/O devices and memory (types and amount) of a microcomputer are chosen as per the specific application. For example, the personal computers usually contain a keyboard and mouse as input devices. Figure 1.5 shows a typical microcomputer system.

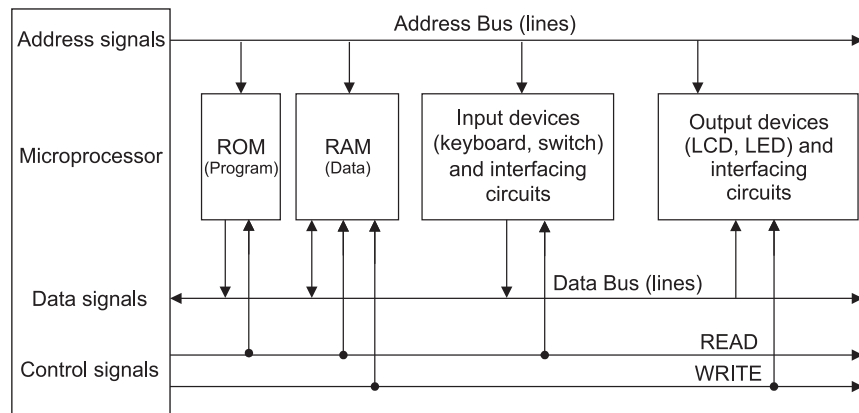


Fig. 1.5 Microcomputer system built around a microprocessor

1.2.3 Microcontroller

The microcontroller is an entire computer built into a single semiconductor chip. The term “micro”, as mentioned above, means small in size, and the term “controller” means they are normally used to control the machines or gadgets. Hence, by definition, microcontrollers are designed for machine control and/or monitoring applications, rather than direct human interactions.

A microcontroller contains all (or major) components in a single chip to make systems based around them standalone. It includes data and code memory, various on-chip peripherals like timers/counters, serial port, A/D converters, D/A converters, etc., interface controllers, and general purpose I/O ports which allow it to directly interface to external environment. The amount and type of memory, I/O and on-chip peripherals varies across wide range of available microcontrollers. The specific microcontroller is chosen based on the requirements of the end application. The block diagram of a microcontroller is shown in Figure 1.6.

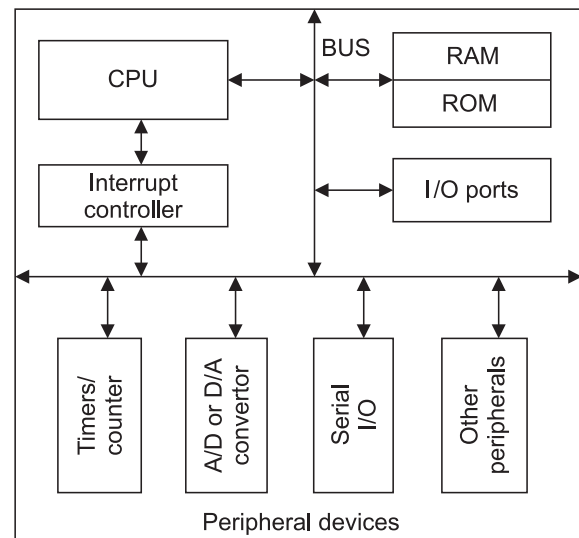


Fig. 1.6 Block diagram of microcontroller

Microcontroller based products are generally physically smaller, more reliable and cheaper than microprocessor based products like personal computers. They are most suitable for applications where cost per unit, size and power consumption are very important factors.

The key features of microcontrollers are that they are embedded within an application or a product (often a consumer product) and are usually designed to perform a few specific control oriented operations, for example, microcontrollers are widely used in modern cars where they will perform dedicated tasks, i.e. to regulate the fuel injection in an engine, to regulate brakes of all wheels, or to control the car's air conditioning, or a microcontroller responsible for the air-bag control. Since microcontrollers are usually embedded inside some other device, they are also referred as embedded controllers.

In conclusion, a microcontroller contains all resources within a chip (on-chip) to satisfy need of an average application or project. This integration of all necessary resources will offer following advantages:

1. On-chip peripherals have smaller access time; hence, speed is more.
2. Less number of chips, less wiring, reduced PCB size which in turn reduces development time, manufacturing cost and increases reliability of the system, which are the key factors in the embedded system design.
3. Easy maintenance and future upgradeability



THINK BOX 1.2

Manufacturing cost and product size are the two most important criteria (factors) in a new product designs. How do microcontrollers help achieve the above criteria?

The microcontroller integrates CPU and all other hardware resources like memory, timers, I/O circuits and other peripherals in a single chip. This will require a small PCB for the final product, moreover, integration of major hardware resources in a single chip will reduce design, development and debugging time which will reduce the price of a final product.

1.2.4 Comparison between Microprocessor and Microcontroller

Based upon above discussion the brief comparison between microprocessor and microcontroller is presented in Table 1.1 with respect to their hardware components, instruction set and applications.

1.3 | CLASSIFICATION OF MICROCONTROLLERS

The microcontrollers/processors can be classified on the basis of word length, memory architecture, core (CPU) architecture and instruction set architecture.

Table 1.1 Comparison between microprocessor and microcontroller.

Microprocessor	Microcontroller
Microprocessor is complete functional CPU, i.e. it contains ALU, registers, stack pointer, program counter, instruction decode and control unit and interrupt processing circuits.	Microcontroller is complete functional microcomputer, i.e. it contains the circuitry of microprocessor and in addition it has built in memory (ROM, RAM), I/O circuits and peripherals necessary for an application.
Microprocessor instruction sets are data processing intensive, means powerful addressing modes and many instructions to move data between memory and CPU to handle large volumes of data.	Microcontrollers have instruction sets that are related to the control of inputs and outputs, means they have many bit handling instructions along with byte processing instructions.
Microprocessor based products are primarily designed to interact with humans and are more flexible to design.	Microcontroller based products are primarily designed to interact with machines; once a system is designed they are less flexible.
Access times for external memory and I/O devices are more, resulting in a slower system.	Access times for on-chip memory and I/O devices are less, resulting in a faster system.
Microprocessor based systems require support devices and are usually bulkier, costly, less reliable and consume more power.	Microcontroller based systems require less external hardware, reducing PCB size and hence are compact, cheaper, more reliable and consume less power.
Software protection is not possible because of the requirement of external code memory.	Software protection is possible because of on-chip code memory.

1.3.1 Word Length: 4, 8, 16, 32, 64-bit Microcontrollers

The number of bits in a binary pattern that a microcontroller understands and processes at a time is called word length, i.e. number of bits on which ALU can perform operations at a time. Microcontrollers are classified according to their word length, it ranges from 4 or 8 bits for small systems, 16 bits for medium systems and 32 or 64 bits for high-speed large systems.

Bus width, internal bus width, register width and bits are the different terms used interchangeably for the term word length. The COP400 family from National, and TMS 1000 from Texas Instruments are 4-bit microcontrollers. 8048 family (MCS 48), 8051 family (MCS 51) from Intel, COP 800 family from National, 6805, 6811 from Motorola, PIC16 from Microchip are all 8-bit microcontrollers. 8051XA, 8096 from Intel, 6812 from Motorola are 16-bit microcontrollers. The Intel 251 family, 683XX, ARM 7, 9, 11 based microcontrollers are 32-bit microcontrollers.

Microcontrollers of MCS 51 family are known as 8-bit microcontrollers as they can operate on 8 bits at a time.

THINK BOX 1.3

What is the advantage of wider word length (or bus width)?

Increased data handling and processing capacity (per machine cycle)

1.3.2 Memory Architecture: Von Neumann and Harvard Architectures

This classification of microcontrollers is mainly based on the organization of a program and data memory. The basic characteristics and comparison between these architectures is presented below.

1. Von Neumann Architecture It has a single memory storage to hold both program instructions and data, i.e. common program and data space. The CPU can either read an instruction or data from the memory one at a time (or write data to memory) because instructions and data are accessed using same bus system. The Von Neumann Architecture is named after the mathematician and computer scientist John Von Neumann. The basic organization of memory in this architecture is shown in Figure 1.7.

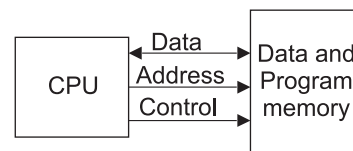


Fig. 1.7 Von Neumann architecture

The advantage of Von Neumann architecture is simple design of microcontroller chip because only one memory is to be implemented which in turn reduces required hardware. The disadvantage is slower execution of a program. It is also referred as Princeton architecture as it was developed at Princeton University. Motorola 68HC11 microcontroller is based on Von Neumann architecture.

2. Harvard Architecture It has physically separate memory storage to hold program instructions and data, i.e. separate program and data space. Since it has separate buses to access program and data memory, it is possible to access program memory and data memory simultaneously. The organization of memory and buses in this architecture is shown in Figure 1.8.

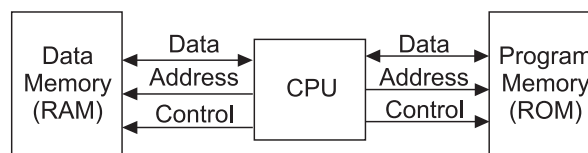


Fig. 1.8 Harvard architecture

The advantage of a Harvard architecture microcontroller is that it is faster for a given circuit complexity because it offers greater amount of parallelism. The disadvantage is that it requires more hardware, because two sets of buses and memory blocks are required. MCS 51 (8051 family) and PIC microcontrollers are based on Harvard architecture.

1.3.3 Core Architecture: Microcoded and Hardwired Designs

This classification of microcontrollers is mainly based on design of execution unit (ALU + Instruction decode and control unit). The term *core* is used collectively for all circuitry responsible for execution of the instructions.

1. Hardwired Design A hardwired microcontroller/processor uses the bit patterns of the instructions to select and activate specific circuits (may be unique to each instruction) to execute the instructions. All control signals (or sequence of steps) required to fetch, decode and execute the instructions are generated and controlled by combinatorial logic and state machine circuitry. Hardwired core design is shown in Figure 1.9.

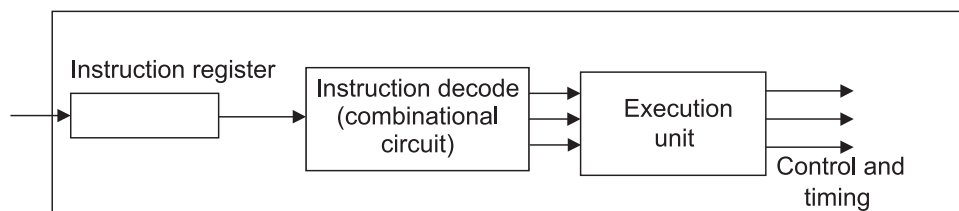


Fig. 1.9 Hardwired core

2. Microcoded Design

Microcode is a group of instructions (usually referred as microinstructions) used to implement the instructions of a microcontroller/processor. It resides in a ROM or a programmable logic array (PLA) that is part of the microcontroller chip. The microinstruction is group of bits (stored in ROM) used to represent the sequence of control signals to fetch,

decode and execute the instruction, i.e. control signals (in a sequence) for every instruction are generated using memory. The microinstructions are at an even more detailed level than machine language.

Every instruction has its own microinstruction sequence (microprogram). When an instruction is placed into the instruction register, a few bits of the instruction will identify address of microcode (or effectively instruction routine) and finally the control signals are generated in a sequence to complete a specified operation. The original 8051 is based on microcoded design, which requires 12, 24 or 48 clock cycles for each instruction to execute. The microcoded core is shown in Figure 1.10.

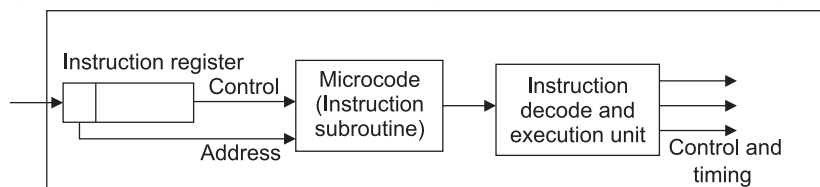


Fig. 1.10 Microcoded core

Both designs have advantages as well as disadvantages. A microcoded core is simpler to design, and can be implemented faster. A hardwired microcontroller is more complex to design because instruction decoder will be complex and difficult to debug and is less flexible. The advantage of hardwired design is that it will execute instructions faster.

1.3.4 Instruction Set Architectures: CISC and RISC

This classification is based on capabilities and type of instructions of a microcontroller.

1. CISC: Complex Instruction Set Computer

In the early days of the computer development, most of the program development work was done in assembly language because high-level languages were not developed. Therefore, CPU designers tried to make instructions that will perform as much work as possible, which in turn has led to development of instructions with many addressing modes for arithmetic, logical and data transfer operations. These powerful instructions provide greater flexibility in performing operations. The CISC architecture requires less number of instructions per program at the cost of number of cycles per instruction. The 8051 microcontroller is based on CISC architecture, for example multiply and divide instructions are complex instructions for which operation is performed in hardware.

The common characteristics of CISC architecture are

- Complex hardware:** complex as well as more addressing modes, variable instruction size
- Many clock cycles to execute an instruction
- High code density—small program size
- Complex data types

2. RISC: Reduced Instruction Set Computer

The simple instructions which perform a few operations at a time will provide high performance because of less hardware requirements for instruction decoder. Thus, instructions require very less time to execute. The RISC instructions have few addressing modes supported by all instructions. It reduces the cycles per instruction at the cost of the number of instructions per program. The Microchip PIC microcontrollers are based on RISC architecture.

The common characteristics of RISC architecture are

- Simple hardware:** simple and less addressing modes, fix instruction size
- Single clock cycle execution, uniform instruction format
- Low code density—larger program size
- Few data types in hardware
- Emphasis is on software: Compiler design is more complex

THINK BOX 1.4



What are the main architectural differences between microprocessors/ controllers?

- Word size (size of the ALU)
- Addressable memory (memory addressing capacity)
- CPU design (core architecture)
- Instruction execution speed (maximum clock speed supported)
- Instruction set (type of instructions, addressing modes)
- Number of registers

1.4 | CHOOSING A MICROCONTROLLER

The microcontrollers are chosen mainly as per requirements of a particular application and a product. The major factors that have to be considered for selecting microcontrollers are discussed briefly in the following section.

(a) Computational Requirements The microcontroller should have enough speed and processing capability to handle all operations of an application in a real time. The speed should be just sufficient to meet the computational needs of an application efficiently. Higher operational speeds than required will unnecessarily increase power consumption.

The microcontroller word length i.e., 8, 16, or 32 bits should match most of the data types to be processed. Hence, it is a major factor in evaluating computational capabilities and suitability of microcontroller for an application.

(b) Hardware Resources Requirement The microcontroller should have sufficient program and data memory to store and execute an application program. An application program should take around 70–80% of the microcontroller's program memory; this will provide space for future upgradations. Microcontroller should have enough I/O pins to provide user interface and connectivity to other modules of an application. It should also have all (or maximum) peripherals like timers/counters, ADC, DAC, serial port, etc., on chip to make product as compact and as reliable as possible.

(c) Power Requirements A microcontroller should have low power consumption. It is critical factor for portable and battery powered products.

(d) Software and Hardware Development Tools and Family Cost and availability of the software development tools like compiler/assembler, debuggers, emulators or Integrated Development Environment (IDE) are important factors to choose a microcontroller. Availability of software libraries and software building blocks will ease the product design. The design team should be well versed with all this tools and family of microcontroller. For example, if design team has experience and expertise in Intel 8051 family, then choosing the 8051 family microcontroller will lead to perfect design of the product. This saves development efforts and reduces implementation time.

The third-party support for all tools is also an important factor to choose a microcontroller.

(e) Cost It is a major factor in selection of a microcontroller. Designer should select cheapest microcontroller that satisfy applications need.

(f) Availability The microcontroller and support chips, if any, should be available easily in enough quantities now and in future throughout life cycle of a product.

(g) Future Upgradeability and Maintenance The microcontroller's ability to upgrade to higher performance or low power versions in future and ease of maintenance of the product also have to be considered.

1.5 | APPLICATIONS OF MICROCONTROLLERS

Microcontrollers have changed the way we live in past few decades. They have entered in almost all aspects of our life. Their production counts are in the billions per year, and are integrated into diversified appliances as discussed below.

1. **Household appliances:** Microwave oven, washing machine, coffee machines, refrigerators, digital cameras, alarm clocks, toys, home security systems, remote controllers, exercise machines, sewing machines, air conditioners, etc.
2. **Office and commercial appliances:** Fax machine, photocopier, scanner or printer machine, intercom, computer systems (discussed below), calculators, ATM machines, CCTV camera and surveillance systems, point of sale systems, weighing scales, elevators, lifts, and many products included in household appliances.
3. **Telecommunication:** Telephones, phone answering machines, mobile phones, satellites, etc.
4. **Entertainment and gaming:** Televisions, VCRs, music players, stereo systems, set-top boxes, play stations, video games, musical instruments, etc.
5. **Automotive industry:** Fuel injection, ABS, ignition, power windows and seats, climate control, air bags, brake control, etc.
6. **Industrial automation and manufacturing:** Motor control systems, data acquisition and supervisory systems, industrial robots, electronic metering, etc.
7. **Electronic measurement instruments:** Digital multi-meters, frequency synthesizers and oscilloscopes, logic analyzers, spectrum analyzers, digital thermometers, tachometers, etc.
8. **Biomedical systems:** ECG recorder, blood-cell analyzers, glucose monitor, patient monitoring systems, etc.

9. **Computer systems:** Keyboard controller, CD drive or hard-disk controller, CRT controller, DRAM controller, printer controller, LAN controller, etc.
10. **Military weapons, guidance and positioning systems.**
11. **Aerospace industry.**

And any automatic or semiautomatic devices around us usually contain microcontrollers.

1.6 | HISTORY AND INTRODUCTION TO THE 8051 MICROCONTROLLER FAMILY

Intel introduced the first microprocessor, the 4004, in 1971. This was the beginning of the microprocessor revolution. The 4004 was a 4-bit microprocessor with 640 bytes of memory addressing capacity and 108 KHz clock. It was developed for an electronic calculator. Later, Intel introduced the 8-bit microprocessors, the 8008 and its upgraded version—the 8080. Today, 64 bit (as well as 128 bit) microprocessors with nearly 3 GHz clock and 1TB addressing capacity are available. Till today Intel is the leading company in developing and manufacturing new microprocessors.

In 1976, Intel introduced its first microcontroller, the 8048. It integrated the processor core with program and data memory, two register banks, 1 timer, 27 I/O lines and 2 interrupts. It has 1KB mask ROM as a program memory and 64 bytes of on-chip RAM. It also has external memory support.

In 1980, Intel introduced the 8051. It was an extension to the 8048. The extensions include extra program and data memory, four register banks, multiply and divide instructions, additional timer, UART and five interrupts.

1.7 | OVERVIEW OF THE 8051 FAMILY

The 8051 is an 8-bit microcontroller, i.e. 8 bit internal data bus width. It is optimized for 8 bit mathematical and Boolean (single bit) operations. Its family includes 8031, 8032, 8051, 8052, 8751 and 8752 microcontrollers. The architecture of the 8051 family of microcontrollers is referred to as the MCS 51 architecture, or simply as MCS51.

1.7.1 Features of the 8051 (MCS 51) Family

The key features of 8051 microcontroller are

1. 8-bit CPU with Boolean processing capabilities
2. 4K bytes on-chip *program memory
3. 128 bytes on-chip data memory
4. 64 Kbytes each program and external data address space
5. 32 bidirectional I/O lines organized as four 8-bit I/O ports
6. Serial port—Full duplex UART
7. Two 16-bit timers/counters
8. Two-level prioritized interrupt structure
9. Direct byte and bit addressability
10. Four register banks
11. Binary or decimal arithmetic support
12. Hardware multiply and divide operations
13. 12 clock cycles per machine cycle

The simplified block diagram of the 8051 showing hardware resources and their organization is shown in Figure 1.11.

The 8051 microcontroller has 4K masked ROM, 128 bytes of RAM, two 16-bit timers, 32 I/O pins, one serial port and five sources of interrupts. Apart from 128 bytes internal RAM, the 8051 has various special function registers (SFRs), which controls on-chip peripherals. Programming of the various on-chip peripherals of the 8051 is achieved by loading the appropriate control words into the corresponding SFRs. The 8031/8032 is similar to the 8051, except it does not contain the on-chip ROM, i.e. they are ROMless devices. As a result, external ROM must be added in a system to store a program. While adding external ROM, two 8 bit ports are engaged (as address/data bus), that leaves only two ports for I/O operations. The 8052/32 has extra 128 bytes of RAM and an extra timer. Since 8051 is subset of 8052, all programs

*Some members do not have on-chip ROM and others have 8K on-chip ROM.

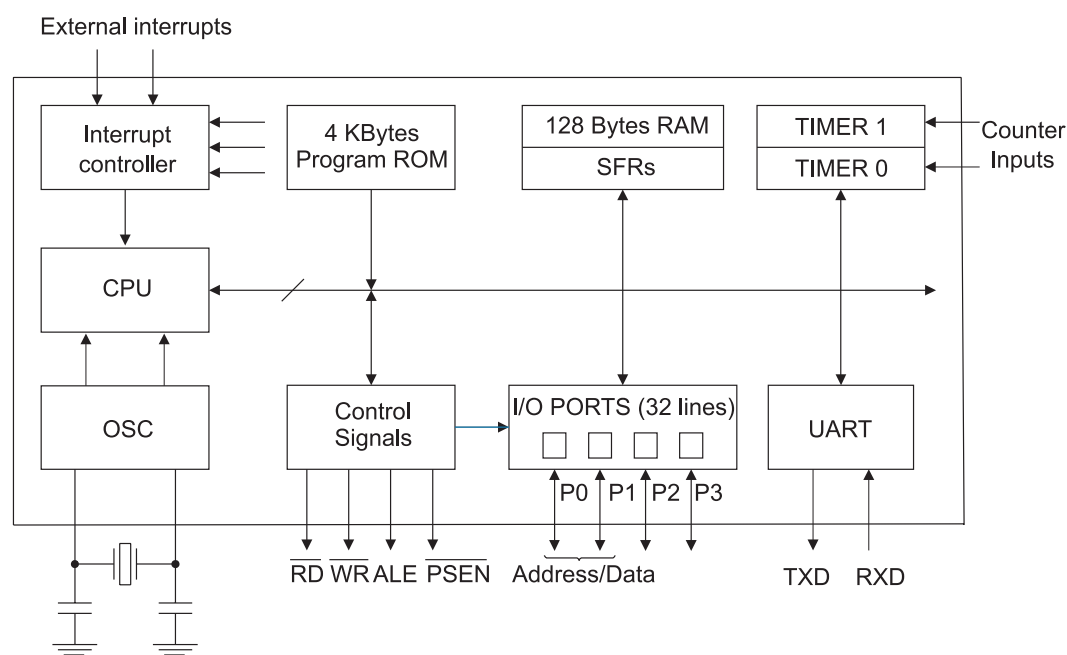


Fig. 1.11 The 8051 block diagram

written for 8051 will run on 8052 but reverse may not be always true. Table 1.2 gives the comparison of hardware resources of MCS 51 family of microcontrollers.

Table 1.2 Comparison of hardware resources of MCS 51 family

Feature	8031	8051	8751	8032	8052	8752
Program memory	None ROMless	4K ROM	4K EPROM	None ROMless	8K ROM	8K EPROM
Data memory	128 Bytes	128 Bytes	128 RAM	256 Bytes	256 Bytes	256 Bytes
Timers/counters (16-bit)	2	2	2	3	3	3
I/O pins	32	32	32	32	32	32
Serial port	1	1	1	1	1	1
Interrupt sources (Reset not included)	5	5	5	6	6	6

Though the 8051 has 8 bit CPU, its instruction set is also optimized for the Boolean (single bit) operations inherently required in a real world control applications. The Boolean processor provides direct support for bit manipulation. This leads to more efficient programs that need to deal with binary operations frequently used in the machine control applications. Bit addressing can be used for test pin monitoring or user defined program control flags.

The design of MCS 51 family microcontrollers listed in Table 1.2 is based on HMOS (High speed Metal Oxide Semiconductor) technology operating at 12 MHz. CHMOS (complementary HMOS) versions of these family members are also available, they are represented by additional letter 'C' in a part number, for example, 80C51, 87C51. The CHMOS devices have the following advantages:

1. Low power consumption
2. High stability (noise immunity)
3. High speed
4. Support power down and idle modes

1.7.2 8051 Variants and Enhancements

The term "Variants" means the 8051 compatible microcontroller. The 8051 has the widest range of variants amongst all microcontrollers in the market because Intel has licenced other manufacturers to design the microcontrollers based on the 8051 core provided that they should remain code compatible with the original 8051. Today there are around thousand

**THINK BOX 1.5****What makes 8051 an 8-bit microcontroller?**

The number of bits that ALU can process at a time i.e. size of the ALU. The 8051 has 8 bit ALU.

However there are few exceptions, one of the most popular example is Intel 8088 microprocessor, though it is having 16 bit ALU, it is referred by Intel as an 8 bit microprocessor because it has only 8 bit external data bus interface!

variants of the 8051 manufactured by more than 20 semiconductor companies. The enhancements contain more memory, different on-chip peripherals and higher operating speeds. The enhanced features are under control of additional SFRs. The variants will differ in various aspects like:

1. **Amount of on-chip memory:** RAM and ROM
2. **Type of memory:** RAM—static or dynamic, ROM—Masked ROM, EPROM, EEPROM, Flash, NVRAM
3. **On chip peripherals:** Timers/counters, I/O ports, UART, ADC, DAC, PWM, I2C, SPI, Capture module, Watch dog timer, Real time clock, Analog comparator, USB, CAN, power management module and many application specific peripherals
4. Operating speeds and oscillator sources
5. Operating voltage
6. Power consumption
7. Implementation technology
8. Package type and number of pins

The leading and popular manufacturers of the 8051 variants are Atmel Corporation, Dallas Semiconductors, and Philips (NXP). The 8051 variants from above manufacturers are discussed in detail in Chapter 22. Their product features are briefly discussed below.

(a) Atmel Corporation Flash memory variants, 20/40 pins, varying operating voltages from 2.7 V to 6 V, low cost. These devices can operate from 0 to 24 MHz. Operating with 0 Hz frequency means the contents of RAM are frozen in absence of clock and they do not require refreshing because of SRAM memory. The low frequency operation is preferred when power consumption is an important consideration, for example, for the battery operated devices. The 20 pin members (89C1051, 89C2051) are preferred when small board (PCB) space of final product is desired (provided that the design requirements are fulfilled with only 15 I/O pins). The other important feature is that they have flash ROM that can be erased and programmed electrically.

(b) Dallas Semiconductors NV RAM (battery backed), flash, EPROM variants, program download through serial port eliminating need of EPROM programmer and need of removing chip from the system, this feature is known as In System Programming (ISP), built in real time clocks for few variants, High-speed variants (single clock per machine cycle).

(c) Philips Corporation (NXP semiconductors) Maximum variants, flash memory variants, high integration of built in peripherals, ISP, different packages and pins, low end variants with fewer features, low operational voltage variants, extended I/O, low cost.

1.7.3 Comparison between MCS 51, PIC, AVR and HCS11/12 Families

There are many families of 8-bit microcontrollers from different manufacturers and each family has a wide variety of variants to meet requirements of different applications. Among the various families, the PIC families from Microchip, AVR family from Atmel and HCS 11/12 family from Motorola (Freescale) are most popular. A brief comparison between some of the members from these families is given in Table 1.3.

1.7.4 Advantages of Using 8051 Family of Microcontrollers

The use of 8051 family microcontrollers offers the following advantages:

1. **Availability and support:** Easily and readily available and widely supported, free and commercial third party support is easily available because they are more popular. Hardware and software development tools and training are easily available and are inexpensive. High level language compilers are also available.

Table 1.3 Comparison between popular 8-bit microcontrollers

Parameters	MCS 51 (8051)	PIC	AVR	HCS11/12
	89C51	PIC18F242	ATmega32	MC68HC912B32
	89C52	PIC18F452	ATmega64	M68HC11K
	89C54	dsPIC30F (16 bit)		
ROM (Flash)	4K	16K	32K	32K
	8K	32K	64K	20K
	16K	Up to 144K		
RAM	128 bytes	768 bytes	2K	1K
	256 bytes	1536 bytes	4K	640 bytes
	256 bytes	8K		
EEPROM	NO	256 bytes	2K	768 bytes
		256 bytes	2K	768 bytes
		4K		
Timers	2(16-bit)	4	3	8
	3(16-bit)	4	4	8
	3(16-bit)	5		
I/O pins	32	34	32	63
		34	53	62
		Up to 54		
Speed up to	33 MHz	40 MHz	16 MHz	25 MHz
		40 MHz	16 MHz	16 MHz
		120 MHz		
Interrupts	5	17	21	24
	6	18	35	22
	6	32		
Power down mode	YES	YES	YES	YES
Watchdog timer	NO	YES	YES	YES
Voltage range	5 V	2 to 5.5 V	4.5 to 5.5 V	6.5 V
				3.0 V to 5.5 V
A to D convertor	NO	10 bit/12 bit	8 CH, 10-bit	8 CH, 10-bit
			8 CH, 10-bit	8 CH, 8-bit
Others	UART	SPI, I2C, PWM In Circuit Debug CAN(dsPIC30F)	JTAG interface, PWM	PWM SPI SCI

2. **Low cost:** High level integration of many peripherals within single chip, only a few external components needed to create a working system.
3. **Effective architecture:** Architecture optimized for the single-bit operations, highly desirable for control applications. Single bit instructions require fewer bytes of code and hence faster execution.
4. **Multiple vendors:** More than 20 manufacturers, more than thousand variants, something for everyone.
5. **Compatibility:** Op-codes are same for all variants, therefore, easy to upgrade to newer variants.
6. **Constant improvement:** Constant improvement in implementation technology and reduction in power consumption.

THINK BOX 1.6



Why are there so many microcontrollers?

These days the microcontrollers are usually designed for specific area of applications. To serve large number of applications (to meet specific requirements of different applications), many microcontrollers with different hardware resource (customized resources) are developed.

1.8 | EMBEDDED SYSTEMS

To understand the term “Embedded System”, let us first try to see dictionary meaning of the word “Embedded”. It means ‘hidden inside’ or ‘fixed’ or ‘implanted in to bigger system’.

The embedded systems are devices made from combinations of a computer (microcontroller/processor) hardware and software programmed for a fixed and dedicated application(s). The device may be part of (or *implanted* in to) larger system, or specific part of an application or a product (often a consumer product). The computers (microcontrollers/processors) are *hidden* in the system. It means, the software that controls the application is permanently fixed in to ROM and is not accessible to user of the device. This is the key difference between embedded systems and general purpose computers which can be configured and programmed as per user’s choice. The most common examples of embedded systems are washing machines, microwave oven, digital clock, fax machine, cell phones, etc...

The key characteristics of embedded systems are the following:

1. They are designed to perform specific (or limited) tasks.
2. They are tightly constrained with respect to power consumption, size, design, testing and manufacturing costs. These constraints are achieved by selecting microcontroller speed just sufficient to satisfy computational needs, limited memory, and limited peripheral resources to achieve design goal.
3. They guarantee the response to events and completion of tasks within specified time. This is more popularly known as real time operation.

The microcontrollers are most important parts of embedded systems; therefore it is necessary to have knowledge of microcontroller architecture, programming and interfacing with real world to design an embedded system. **This book is written to serve this purpose.**

Embedded Microcontrollers

When all resources (peripherals + memory) required for an application are available within microcontroller chip, it is called an embedded microcontroller. The only additional requirements are power supply, clock and preferably reset circuit.



THINK BOX 1.7

Which architectural techniques are used in newer microprocessor/ controller designs for improved performance?

- Parallel processing
- Co-processing
- Wider buses
- Cache memory
- Pipelining

POINTS TO REMEMBER

- ◆ The basic operations performed by the computers are storing and retrieving binary numbers, arithmetic and logical operations on the binary numbers.
- ◆ The major components of a digital computer system are CPU, memory, I/O unit and system bus.
- ◆ The major components of a CPU are ALU, Instruction decode and control unit, registers, program counter, instruction register and interrupt controller.
- ◆ The address, data and control buses are required for communication between blocks of a computer system.
- ◆ Microcontroller is an entire computer built into a single semiconductor chip.
- ◆ Microcontroller-based systems are compact, fast, easy to design and debug, cheaper and consumes less power.
- ◆ Microcontrollers/processors can be classified on the basis of Word length, memory architecture, core architecture and instruction set architecture.
- ◆ Complex hardware, more addressing modes, variable instruction size and high code density are the key features of the CISC architecture.
- ◆ Simple hardware, less addressing modes, fixed instruction size, single clock cycle execution and uniform instruction format are the key features of the RISC architecture.

- ◆ The factors affecting microcontroller selection are computational requirements, hardware resources and power requirements, cost, availability, future upgradeability and ease of maintenance.
- ◆ The 8051 is an 8-bit microcontroller, i.e. 8-bit internal data bus width optimized for 8-bit math and single bit Boolean operations.
- ◆ The variants of 8051 differ in various aspects like on chip memory, on-chip peripherals, operating speeds and oscillator sources, operating voltage and power consumption.

OBJECTIVE QUESTIONS

1. A microcontroller with 64 KB of code memory will require a program counter of,
(a) 8-bit size (b) 16-bit size (c) 64-bit size (d) none of the above
2. An 8-bit microprocessor has essentially,
(a) 8-bit ALU (b) 8-bit Accumulator
(c) most registers are 8-bit registers (d) all of the above
3. In a microcontroller, execution speed of a given program depends upon,
(a) on-chip/off-chip program memory. (b) number of address lines.
(c) size of the program counter (d) clock frequency
4. The microcontroller is useful in systems that have fixed programs for dedicated applications.
(a) True (b) False
5. Microcontrollers usually have,
(a) CPU (b) RAM+ROM (c) peripherals (d) all of the above
6. In RISC architecture, instructions are simpler in order to,
(a) reduce the hardware complexity (b) enhance clock frequency
(c) reduce compiler complexity (d) all of the above
7. CISC machines,
(a) have fewer instructions than RISC machines (b) use more RAM than RISC machines
(c) have medium clock speeds (d) use variable size instructions
8. RISC machines typically,
(a) have high capacity on-chip cache memory (b) have fewer registers than CISC machines
(c) are less reliable than CISC machines (d) execute 1 instruction per clock cycle
9. Harvard architecture has,
(a) separate data and program memory (b) unified cache memory
(c) multiple functional units (d) an on-chip cache
10. What is the difference between the 8031 and the 8051?
(a) the 8031 has no interrupts (b) the 8031 is ROMless
(c) the 8051 is ROM-less (d) the 8051 has 64 bytes more memory
11. A microcontroller is also referred as computer on a chip.
(a) True (b) False
12. An embedded microcontroller usually means,
(a) a microcontroller for the embedded system
(b) microcontroller with embedded processor
(c) a microcontroller with external memories storing the embedded software
(d) microcontroller with all necessary resources available on-chip
13. The basic operations performed by a computer are _____.
(a) arithmetic operations (b) logical operations (c) data movements (d) all of the above
14. Which of the following are output devices?
(a) Keyboard (b) printer (c) display screen (d) scanner
15. _____ address lines are required to address 2K X 8 bit memory.
(a) 11 (b) 14 (c) 10 (d) 16

16. _____ can understand the difference between instructions and data.
 (a) input/output device (b) memory
 (c) microprocessor/controller (d) system bus
17. The variants of 8051 microcontroller differ in,
 (a) amount of on-chip memory (b) operating speed
 (c) on-chip peripherals (d) all of the above
18. Embedded systems are tightly constrained with respect to,
 (a) power consumption (b) size (c) cost (d) all of the above
19. Microcontroller based systems are usually,
 (a) compact (b) less power hungry (c) very much expensive (d) slower
20. Signals required for memory devices are,
 (a) address lines (b) control lines (c) data lines (d) all of the above
21. _____ is ROM less variant of the 8051.
 (a) 8031 (b) 8051 (c) 8751 (d) 8052
22. The most common examples of embedded systems are,
 (a) washing machines (b) cell phones (c) digital clock (d) all of the above

Answers to Objective Questions

- | | | | | | | | | |
|--------------|---------|---------|---------|--------------|-------------|---------|---------|---------|
| 1. (b) | 2. (d) | 3. (d) | 4. (a) | 5. (d) | 6. (a), (b) | 7. (d) | 8. (d) | 9. (a) |
| 10. (b) | 11. (a) | 12. (d) | 13. (d) | 14. (b), (c) | 15. (a) | 16. (c) | 17. (d) | 18. (d) |
| 19. (a), (b) | 20. (d) | 21. (a) | 22. (d) | | | | | |

REVIEW QUESTIONS WITH ANSWERS

1. **List the basic components of a computer system.**
 A. CPU, memory, I/O unit and system bus.
2. **What is the function of an instruction decoder?**
 A. It interprets the instruction present in an instruction register and determines control signals to be generated to execute an instruction.
3. **What information is stored in memory?**
 A. It stores program instructions, data on which operation are performed and temporary results.
4. **List the signals required by memory devices.**
 A. Address, data and control signals. Control signals also include chip select signal(s).
5. **How many memory locations can be addressed by 20 address lines?**
 A. $2^{20} = 1048576$ locations, numbered from 0 to 1048575. It is commonly referred as 1Mega locations.
6. **What is meant by a term 'general purpose' used with microprocessors?**
 A. The user can decide the tasks to be performed by a system as per his/her requirements (of course, within its capability and limitations!)
7. **Microcoded and hardwired designs are classified with respect to _____.**
 A. Core design.
8. **What is the key feature of CISC instruction set?**
 A. Powerful instructions that will do as much work as possible.
9. **What is meant by 8-bit CPU?**
 A. CPU can recognize and process 8 bits at a time.
10. **Which register holds the address of the next instruction to be executed?**
 A. Program counter.
11. **Why ROM-less versions of microcontrollers exist?**
 A. ROM-less versions are used to develop prototype of an application.
12. **"Data bus must be bidirectional." Justify the statement.**
 A. Because it has to write as well as read data to/from memory or peripherals.
13. **For a battery-driven product, what is the most important factor in choosing microcontroller and other components?**

- A. Power consumption
- 14. List microcontroller-based embedded products attached to the PC.**
- A. Mouse, keyboard, disk drives, CRT drivers, printers.
- 15. Which version of the 8051 is suitable for mass production?**
- A. 8051. It does have 4K masked ROM, which is programmed by manufacturer while production of a chip, which is very much cost effective.
- 16. What is the size of internal memory in the 8051?**
- A. 128 bytes of RAM, 21 special function registers (not discussed yet) and 4Kbytes of ROM
- 17. List the additional hardware components commonly available in a microcontroller.**
- A. On-chip memory (RAM and ROM), I/O circuits, Timers/Counters, Interrupt and timing circuits, etc.
- 18. Why is ROM named so?**
- A. They can only be read by program instructions. They cannot be written by program instructions. Additional hardware support is required to write to them.
- 19. What is the key feature of a Harvard architecture?**
- A. It has separate program and data memory and their signals, allowing the parallel access to both of them, which improves the system performance (by providing parallel processing), i.e. fetching of one instruction (from code memory) and execution of the other may be performed simultaneously.
- 20. How does the size of an on-chip ROM matters?**
- A. Larger on-chip ROM allows complex and larger programs to be written allowing more (and complex) features supported by the product without having extra memory connected. It may also allow future upgradation easily if there is spare on-chip memory.

EXERCISE

1. Discuss the differences between microprocessors and microcontrollers.
2. List the applications of microcontrollers.
3. List the different on-chip resources available in microcontrollers.
4. Compare hardwired and microcoded designs of a microcontroller. Discuss their advantages and disadvantages.
5. Justify statement "CISC instructions offers higher code density".
6. How Boolean processing capabilities of the 8051 are most suitable for the control applications?
7. List the features of the 8051 microcontroller. Compare different family members with respect to available on-chip resources. Why do different family members exist?
8. List the various aspects by which microcontroller variants differ.
9. Discuss the importance of having larger on-chip RAM.
10. What are the advantages of using 8051 family of microcontrollers?
11. Discuss in detail factors affecting choice of microcontroller for a given application.
12. List the differences among various 8051 family members.
13. "The microcontroller's word length, i.e. 8, 16, or 32 bits should match most of the data types to be processed in an application." Justify.
14. How does Von Neumann architecture simplify design of a system?
15. Explain how RISC architecture improves the performance of a microcontroller?
16. What are the different ways of classifying the microcontrollers?
17. For RISC based microcontrollers, compiler design is more complex. How?
18. What is in-system programming? What is its advantage?
19. "Higher operational speeds of microcontrollers than required will unnecessarily increase the power consumption of a system". How?
20. Make a list of manufacturers of microcontrollers and list different microcontrollers made by them along with features and on-chip resources.

Programming Model and Architecture of the 8051

Objectives

- ◆ Create an overview and significance of the architecture and programming model
- ◆ Discuss on-chip RAM address space organization in the 8051
- ◆ Discuss program memory (ROM) and external RAM address space organization in the 8051
- ◆ List the special function registers (SFRs) of the 8051 and their functions
- ◆ List and discuss the importance of flags
- ◆ Show the significance of the bit addressability
- ◆ Introduce the concept of the stack and its use

Key Terms

- | | | |
|-----------------------|--------------------------------|------------------------------|
| • Accumulator | • On-chip/Off-chip Memory | • R0 to R7 |
| • Bit addressability | • Peripheral Control Registers | • Register Banks |
| • Data memory | • Peripheral Data Registers | • Special Function Registers |
| • Data pointer | • Program Counter | • Stack |
| • Flags | • Program Status Word | • Stack Pointer |
| • Memory Organization | • Program/Code Memory | • Status Register |

2.1 | THE 8051 ARCHITECTURE

The architectural block diagram of the 8051 is shown in Figure 2.1, it shows organization of all hardware components and data-path connections between them. It includes 8-bit ALU along with Boolean processing capabilities, program and data memory, four 8-bit I/O ports, two timers/counters, UART, timing and control circuits and oscillator circuit.

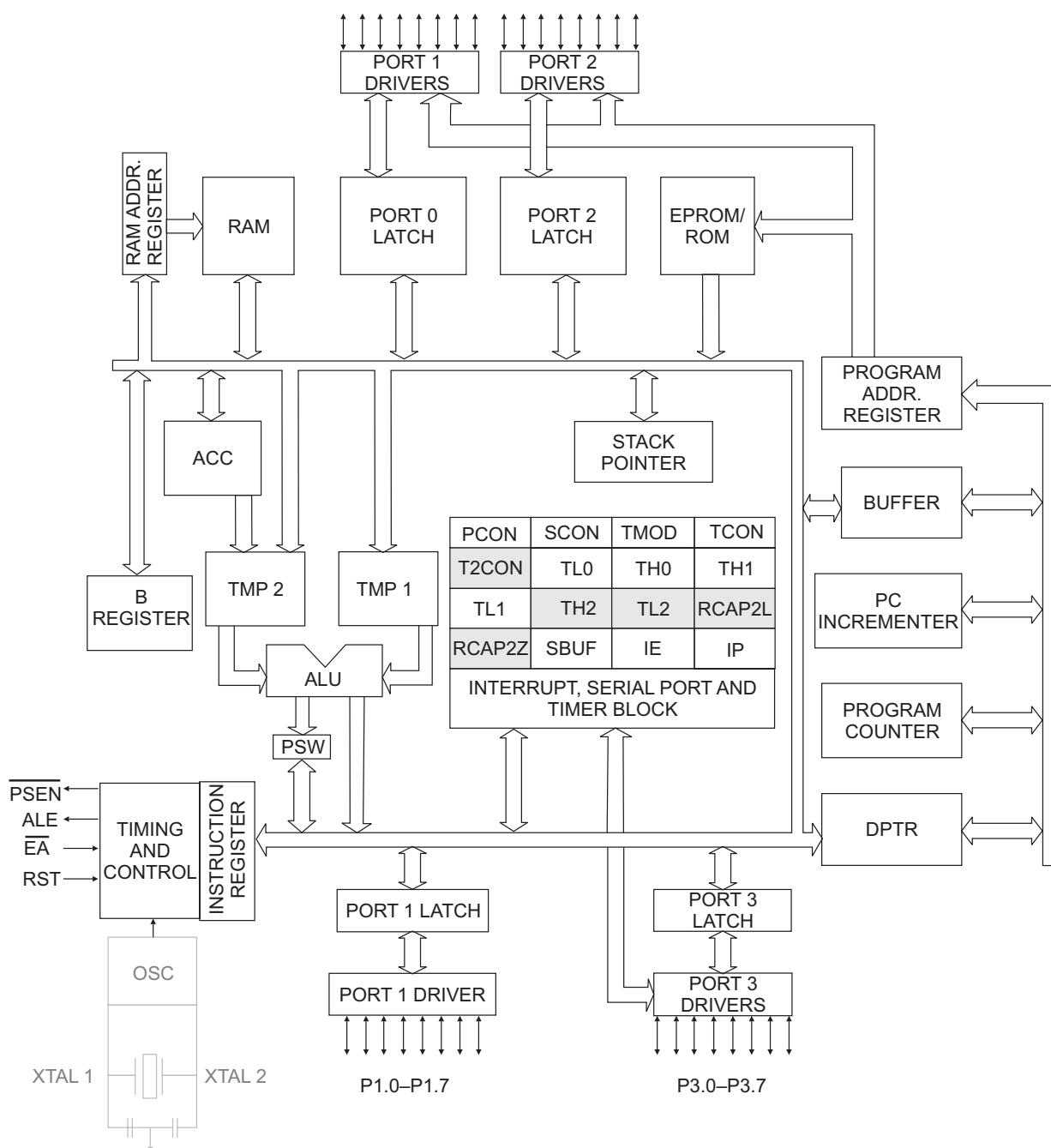


Fig. 2.1 Block diagram of the 8051 microcontroller

1. ALU

The Arithmetic and Logic Unit (ALU) performs all arithmetic (addition, subtraction, multiplication and division) and logical (AND, OR, NOT, EXCLUSIVE-OR and rotating) operations on 8-bit data, i.e. the 8051 has 8-bit ALU. The ALU also updates information about the nature of the result in the flag register (PSW).

2. Memory

The 8051 family has separate on-chip program and data memory. The program instructions are stored in a program memory (ROM/EPROM/EEPROM/Flash based on a family member). The amount and type of on-chip program memory is the key factor that differentiates all the members of the family, for example, 80C51 has 4Kbytes of on-chip ROM, whereas 80C52 has 8Kbyte (ROM), 87C51 has 4Kbytes (EPROM) and 87C52 has 8Kbytes (EEPROM) of on-chip program memory. Total program memory (including on-chip ROM) that can be connected with the 8051 is 64Kbytes. Similarly, data memory can be on-chip or off-chip. Internal data memory (RAM) in 80C51 is 128 bytes and in 80C52 is 256 bytes. There are also on-chip RAM locations which are used to program and control various on-chip hardware peripherals and features of the 8051. They are known as *Special Function Registers (SFRs)*. These memories are discussed in detail in the next section.

3. Peripherals

The 8051 has two 16-bit timers (8052 has three timers) that are used for timing and counting applications. It has full duplex serial port (UART) to handle serial data transmission and reception.

4. Timing and Control Unit

This unit generates all timing and control signals necessary for the execution of instructions and synchronizes all internal activities with the clock.

5. Oscillator

The 8051 has an internal (on-chip) oscillator circuit (partial circuit) which generates the clock pulses by which all internal operations are synchronized. The external resonant circuit is connected with this internal on-chip oscillator circuit to make a complete oscillator. Normally quartz crystal is used to make oscillator functional. Typically, 12 MHz (or 11.0592MHz to support standard baud rates for serial port) crystal is used.

All other blocks of the 8051 are discussed in detail in the subsequent chapters.

2.2 | PROGRAMMING MODEL OF THE 8051

Programming model is programmer's view of a microcontroller/processor. It shows only those components (memory or registers) which can be accessed (read/write) by a programmer along with their internal organization. It is a collection of internal registers (and memory locations) that can be used by a programmer to develop any software (i.e. control and application programs) and to use several features of a particular microcontroller. As these registers and memory locations are used by software instructions, it is necessary to have knowledge of the programming model before we start developing any program.

The programming model of the 8051 is shown in Figure 2.2. It contains 8 (or 16) bit registers and memory locations. Each register (or memory location) has an internal 1 byte address with exception of program counter. Some registers are byte as well as bit addressable, i.e. whole byte of data stored in a register can be accessed (read/write) at a time or individual bits can be accessed at a time. The next section provides overview of all components of the programming model.

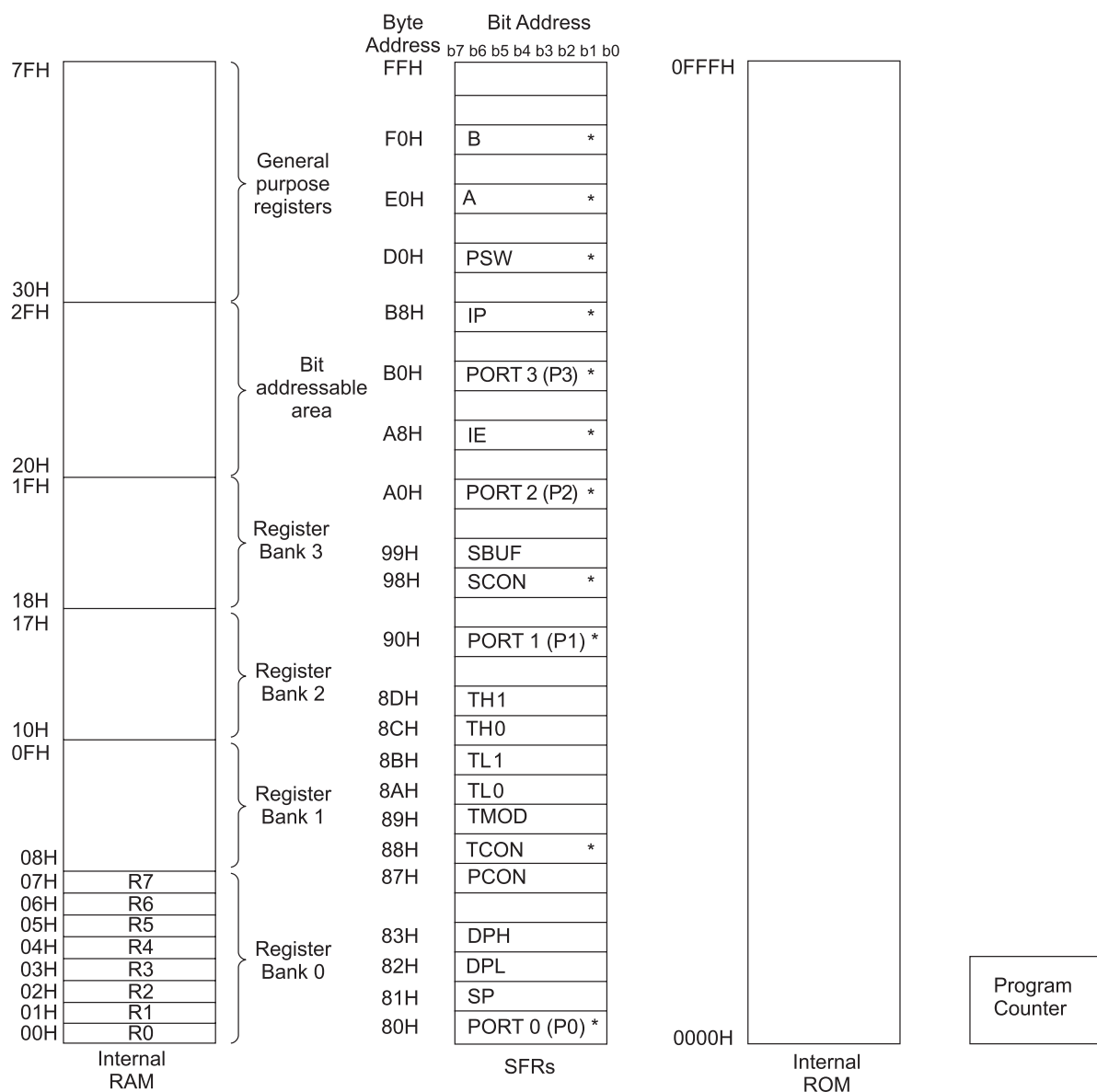
THINK BOX 2.1



How does a microcontroller's block diagram differ from the programming model?

The block diagram shows hardware architectural components like logic blocks (circuits or devices) used for data handling and processing. It also shows how these blocks are interconnected. It helps in the hardware design of a system.

The programming model shows only those components (memory or registers) which can be accessed (read/write) by a programmer. It helps to learn and develop software for a microcontroller.



*Indicates the SFRs which are also bit addressable

Fig. 2.2 Programming model of the 8051

2.3 ON-CHIP MEMORY ORGANIZATION

The 8051 On-chip memory is organized into three general categories; Special Function Registers, internal RAM and internal ROM as shown in Figure 2.2.

1. Special Function Registers (SFRs)

These registers are used to program and control various on-chip hardware peripherals and features of the 8051. Each SFR has a name which specifies the purpose of the SFR. For example, TCON (Timer Control) register is used to control timer

The remaining chapter is not given in this PDF. For more details.
<http://www.mhhe.com/patel/mbes>

Program Development Process and Tools

Objectives

- ◆ Discuss and compare various programming languages
- ◆ Introduce the assembly language structure
- ◆ Develop a simple assembly language program
- ◆ Discuss qualitatively the steps involved in the program execution
- ◆ Discuss the software and the hardware development tools
- ◆ Discuss the significance of the documentation tools
- ◆ Show the significance and use of Integrated Development Environment in the system development
- ◆ Introduce the common assembler directives
- ◆ Explain in detail the program development cycle
- ◆ Show how to load a program in to a microcontroller
- ◆ Describe the Intel hex file format

Key Terms

- | | | |
|---------------------|-----------------------|--------------------|
| • Assembler | • Flowcharts | • Machine Language |
| • Assembly Language | • Hex File | • Mnemonic |
| • Comment | • High-level Language | • Object File |
| • Compiler | • IDE | • Op-codes |
| • Debugger | • Instruction | • Operands |
| • Documentation | • Label | • Pseudo-codes |
| • Download/Burning | • Linker | • Simulator |
| • Emulator | • Logic Analyzer | • Source Code |

The first step in *learning any new language* is to start with most common and simple words and grammar or rules of their usage. Applying the same logic to the learning process of an assembly language, we start with the common terminology and structure of an assembly language and then the complete program development process beginning from program statement to loading a program in to microcontroller memory is discussed. This process is described with the help of a simple program example.

3.1 | PROGRAMMING LANGUAGE

Programming means developing a sequence of commands (or instructions) that can be used by computer system to perform a predefined task. It also involves troubleshooting or debugging of instruction sequence to ensure that the desired operation is performed. There are three types (levels) of programming languages based on how closely the language statements are related and resemble to actual operations performed by the microcontroller/ processor. The three types of languages are Machine language, Assembly language and High-level Language.

3.1.1 Machine Language

The microcontroller/processor is a digital device which understands and operates on binary digits 0 and 1, stored as voltage levels in the circuits. The microcontroller needs commands in form of bit patterns (of 0s and 1s) to perform any operation. These binary bit patterns are called *instructions* or *machine codes* which are recognized and processed by a microcontroller to accomplish a task. The most common operations are storing and retrieving binary data, arithmetic and logical operations. Each microcontroller has its own instructions, meanings, and syntax. The designer of the microcontroller decides these bit patterns or instructions based on the number and type of operations it is required to perform. The entire set of the binary instructions is called *instruction set* and it describes and represents machine language of particular microcontroller. Every instruction is represented by a unique bit pattern to distinguish it from another instruction. Since the microcontroller understands only bit patterns, every operation has to be specified only in a machine language.

Single instruction may not be sufficient to accomplish a task; therefore, we need group of instructions written in a particular sequence. These instructions written in a sequence to perform a task are collectively known as a *program* or a *microcontroller program*.

The number of bits in a binary pattern that a microcontroller recognizes and processes at a time is called word, and microcontrollers are classified according to their word length, it ranges from 4 bits for small systems to 64 bits for high-performance computers. Microcontrollers of MCS 51 family are known as 8-bit microcontrollers as they can operate on 8 bits at a time.

3.1.2 Assembly Language

The microcontroller understands only machine language, therefore, we must specify program only in a machine language, however it is not suitable for human use because it is difficult to write and understand instructions using patterns of 0s and 1s. Also, to deal with patterns of 0s and 1s is quite tedious, inconvenient and error inductive for most humans.

For convenience, one can represent the binary instructions in Hexadecimal codes, which is compact form of binary numbers. This will be improvement over dealing directly in binary numbers because instructions can be represented in a compact form and because of that chance of making mistake reduces.

Even if the instructions are written using hexadecimal codes, it is still difficult and time consuming to develop and understand programs. Therefore, manufacturer of microcontroller/ microprocessor assigns unique English-like word (code) to all binary instructions which are referred as *mnemonics*. The mnemonic for an instruction is a word (usually three or four letters) that indicates the operation to be performed by that instruction. The entire group of instructions when represented as mnemonics is called *assembly language of the microcontroller*. It is easy and convenient to develop and understand assembly-language programs. Every microcontroller/processor has different and specific assembly-language, hence, assembly-language program written for one microcontroller will not work on other. Assembly language is also called low-level language because it directly deals with internal hardware of the microcontroller and its statements directly resembles to the operations performed by microcontroller/processor. A computer program known as *assembler* is used to convert assembly-language programs into machine language.

3.1.3 High-Level Language

High-level languages are named so because programmer need not know and worry about the internal details and architecture of the microcontroller/processor to develop the programs, moreover, their statements represent directly the

program logic (algorithm) rather than actual operations performed by a microcontroller, as a result they are also machine independent. For example, BASIC, Pascal, C, C++ and Java are high-level languages. A computer program known as *compiler* is used to convert high-language program into machine-language program.

3.1.4 Comparison between Programming Languages

A brief comparison between the programming languages is given in Table 3.1.

Table 3.1 Comparison between programming languages

Sr. No.	Machine Language	Assembly Language	High-level Language
1	The operations are specified by binary bit patterns.	The operations are specified by mnemonics.	Simple statements specify the operations.
2	The program development is difficult, inconvenient and error prone.	The program development is simpler and easier than machine language.	The program development is easiest.
3	It is not user-friendly.	It is less user-friendly.	It is more user-friendly.
4	High code density (Less memory required to store a program)		Low code density (More memory required)
5	Faster execution		Slower execution
6	Both languages are microcontroller dependent and require knowledge of internal details of microcontroller to develop a program.		No or very little knowledge of microcontroller is required

The comparison made in the above table is proved using examples in topic 12.12. 'Performance comparison between assembly and C programs'

As an illustration, a sample program to add two numbers is shown in Figure 3.1 for all three languages for a comparison.

Machine Language (8051)	Assembly Language (8051)	High-level Language (C)
01111000 00001010 (78 0A H)	MOV R0,#10	char a =10, b =20;
01110100 00010100 (74 14 H)	MOV A,#20	a = a + b;
00101000 (28 H)	ADD A, R0	

→
Easy to develop and understand

Fig. 3.1 Sample program in different programming languages

The comparison of these programming languages is shown in pictorial form in Figure 3.2.

The programming of the 8051 in a high-level language (C) and their advantages are discussed in detail in Chapter 12 (The 8051 programming in C)

3.1.5 Why Assembly Language?

We need to use and study assembly-language programming because of the following advantages offered by them:

1. The assembly language programs provide direct and accurate control of the microcontroller/processor resources (memory, ports, peripherals and hardware present in a system). The assembly-language programs (for device drivers for peripherals) are efficient in terms of memory requirements and speed of execution.
2. The assembly language codes usually require less memory and they execute faster.
3. It directly allows exploiting the specific features of microcontroller/processor.
4. There is less number of restrictions or rules as compared to the high-level language.

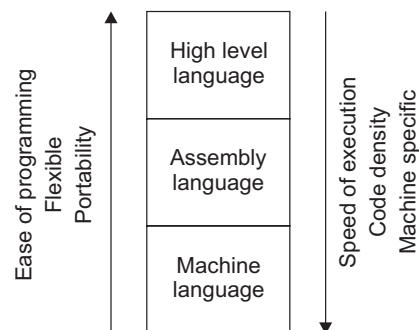


Fig. 3.2 Comparison of programming languages

The remaining chapter is not given in this PDF. For more details:
<http://www.mhhe.com/patel/mbes>



Salient features:

- Logical sequencing of the topics, concise and to the point discussion.
- Pictorial explanation of majority of concepts followed by examples for ease of understanding
- Step by step approach for the software development
- Simulation methods and snapshots of the output for some key examples
- Programming examples in assembly and C language.
- Timing and data flow diagrams for instruction execution.
- Advanced and complex topics like interrupt handling, interrupt latency, lookup tables, timing analysis, stack operations, multiprocessor communications, 8051 enhancements and variants, internal port structure are covered with more clarity.
- All Instructions are explained through use of examples.

FREE ONLINE LEARNING CENTRE FOR THIS BOOK

<http://www.mhhe.com/patel/mbes>

The book is supplemented with separate online resources for Instructors and Students.

Online resources for the Instructors:

- Complete solution manual of the book
- Chapter wise power point slides
- Additional material on Advanced microcontrollers

Online resources for the Students:

- **Lab manual** : A complete lab manual containing 14 laboratories with example references from the book, sample program followed by laboratory exercise to reinforce the concepts
- **Question Papers of different universities with the solution**
- **Additional question bank**
- **Chapter wise objectives, Key-terms and Points to remember**
- **Projects given in the book:** Each project includes problem statement, complete schematic diagram, program development, assembly and/or C program and suggested modifications.
- **Chapter wise Interfacing diagrams.**
- Tutorial of Keil μ Vision4.0 Integrated development environment
- Excellent pedagogy
 - Learning objectives and key terms at the beginning of each chapter
 - Points to remember at the end of each chapter
 - Discussion questions within the topics: **25**
 - Review questions with answers: **310**
 - Exercise questions: **410**
 - Programming examples (Assembly and C): **325**
 - Objective questions: **301**
 - Think boxes with answers: **95**
 - Illustrations (Figures and Tables): **350**
- Coverage of many variants, peripheral devices, PIC and AVR microcontrollers
- Advance serial data transfer protocols: I2C and SPI

The 8051 MICROCONTROLLER Based EMBEDDED SYSTEMS

This book introduces fundamental hardware, software and architectural aspects of microcontroller-based embedded systems in an elementary and integrated manner, providing a strong foundation for the development of expertise in designing such systems. The contents of the book are class tested to ensure the treatment is logical and easy to understand for the fresher.

Salient Features:

- ❖ Simple and easy-to-understand language supported with self-explanatory diagrams
- ❖ Logical sequencing of topics, concise and to-the-point discussion with clarity and simplicity
- ❖ Step-by-step approach for software development
- ❖ Hands-on projects and latest advancements in the field like I2C, SPI, etc., not given in any book
- ❖ Simulation methods and output snapshots for some key examples
- ❖ Programming examples in assembly and C languages; all instructions explained through use of examples
- ❖ Timing and data-flow diagrams for instruction execution
- ❖ Tutorial of Keil Vision4.0 Integrated development environment

<http://www.mhhe.com/patel/mbes>

Write to us at
info.india@mheducation.com

www.mheducation.co.in

**Mc
Graw
Hill
Education**

**Higher
Education**

ISBN-13: 978-93-3290-125-4

ISBN-10: 93-3290-125-2



9 789332 901254